

Universidad de Monterrey



Integración de aplicaciones computacionales

Ejercicio Guiado 4

Microservicios en Flask con JWT

Maestro: Dr. Raúl Morales Salcedo

Nombre: Margarita Concepción Cuervo Citalán #581771

Carrera: ITC, 9no Semestre

Día y hora del grupo de la clase: lunes y jueves 10:00h

21 de septiembre de 2025.

Doy mi palabra de haber realizado esta actividad con integridad académica.

ÍNDICE

Introducción	2
Desarrollo	2
Consideraciones importantes	57
Conclusión	58

Introducción

Este ejercicio guiado tiene como objetivo principal el diseño e implementación de una arquitectura de microservicios, enfocada en la creación de un sistema de gestión para un catálogo de libros. El proyecto se desarrolló sobre una máquina virtual en Google Cloud Platform, donde se configuró un microservicio backend utilizando el framework Flask en Python. Para la autenticación y seguridad, se implementó el estándar de tokens Web JSON (JWT), lo cual permitió proteger las rutas de la API. El cliente, una aplicación web simple desarrollada con HTML, CSS y JavaScript, se comunicó con el backend para realizar las operaciones de consulta y manipulación de datos (CRUD), demostrando así la integración segura y funcional entre servicios.

Desarrollo

Se empezó este ejercicio por acceder a GCP e instalar un sistema de gestión de bases de datos relacionales, para el cual se escogió el de MariaDB. Tras instalarlo, se creó una base de datos, con ayuda de la IA, llamada *Libros* la cual contiene las tablas *Editorial*, *Autor*, *Categoría* y *Libro*. Cada tabla tiene insertados 20 registros que fueron proporcionados también por la IA.

```
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| Libros   |
| information_schema |
| mysql    |
| performance_schema |
+-----+
4 rows in set (0.001 sec)

MariaDB [(none)]> USE Libros;
Database changed
MariaDB [Libros]> CREATE TABLE Editorial (
  ->   id_editorial INT PRIMARY KEY,
  ->   nombre VARCHAR(100) NOT NULL,
  ->   pais VARCHAR(50),
  ->   año_fundacion INT
  -> );
Query OK, 0 rows affected (0.016 sec)

MariaDB [Libros]> SHOW TABLES;
+-----+
```

```
root@maquina01:~  
+-----+  
| Tables_in_Libros |  
+-----+  
| Editorial |  
+-----+  
1 row in set (0.001 sec)  
  
MariaDB [Libros]> CREATE TABLE Autor (  
-> id_autor INT PRIMARY KEY,  
-> nombre VARCHAR(100) NOT NULL,  
-> nacionalidad VARCHAR(50),  
-> fecha_nacimiento DATE  
-> );  
Query OK, 0 rows affected (0.007 sec)  
  
MariaDB [Libros]> CREATE TABLE Categoria (  
-> id_categoria INT PRIMARY KEY,  
-> nombre VARCHAR(50) NOT NULL,  
-> descripcion TEXT  
-> );  
Query OK, 0 rows affected (0.007 sec)  
  
MariaDB [Libros]> CREATE TABLE Libro (  
idbn VARCHAR(20) NOT NULL,  
titulo VARCHAR(150) NOT NULL,  
id_autor INT NOT NULL,  
id_categoria INT NOT NULL,  
id_editorial INT NOT NULL,  
anio_publicacion INT NOT NULL,  
-> id_libro INT PRIMARY KEY AUTO_INCREMENT,  
-> isbn VARCHAR(20) NOT NULL,  
-> titulo VARCHAR(150) NOT NULL,  
-> id_autor INT NOT NULL,  
price DECIMAL(10,2) NOT NULL,  
stock INT NOT NULL,  
formato VARCHAR(50) NOT NULL,  
FOREIGN KEY (id_autor) REFERENCES Autor(id_autor),  
FOREIGN KEY (id_categoria) REFERENCES Categoria(id_categoria),  
FOREIGN KEY (id_editorial) REFERENCES Editorial(id_editorial)  
);  
-> id_categoria INT NOT NULL,  
-> id_editorial INT NOT NULL,  
-> anio_publicacion INT NOT NULL,  
-> price DECIMAL(10,2) NOT NULL,  
-> stock INT NOT NULL,  
-> formato VARCHAR(50) NOT NULL,  
-> FOREIGN KEY (id_autor) REFERENCES Autor(id_autor),  
-> FOREIGN KEY (id_categoria) REFERENCES Categoria(id_categoria),  
-> FOREIGN KEY (id_editorial) REFERENCES Editorial(id_editorial)
```

```
root@maquina01:~  
-> FOREIGN KEY (id_editorial) REFERENCES Editorial(id_editorial)  
-> );  
Query OK, 0 rows affected (0.014 sec)  
  
MariaDB [Libros]> SHOW TABLES;  
+-----+  
| Tables_in_Libros |  
+-----+  
| Autor |  
| Categoria |  
| Editorial |  
| Libro |  
+-----+  
4 rows in set (0.000 sec)  
  
MariaDB [Libros]> INSERT INTO Editorial VALUES  
-> (1, 'Penguin Random House', 'Estados Unidos', 1925),  
-> (2, 'HarperCollins', 'Estados Unidos', 1989),  
-> (3, 'Simon & Schuster', 'Estados Unidos', 1924),  
-> (4, 'Macmillan Publishers', 'Reino Unido', 1843),  
-> (5, 'Hachette Livre', 'Francia', 1826),  
-> (6, 'Grupo Planeta', 'España', 1949),  
-> (7, 'Alfaguara', 'España', 1964),  
-> (8, 'Tusquets Editores', 'España', 1969),  
-> (9, 'Fondo de Cultura Económica', 'México', 1934),  
-> (10, 'Anagrama', 'España', 1969),  
-> (11, 'Editorial Norma', 'Colombia', 1960),  
-> (12, 'Editorial Sudamericana', 'Argentina', 1939),  
-> (13, 'Santillana', 'España', 1960),  
-> (14, 'Siglo XXI Editores', 'México', 1965),  
-> (15, 'Seix Barral', 'España', 1911),  
-> (16, 'Editorial Porrúa', 'México', 1900),  
-> (17, 'Editorial Trillas', 'México', 1959),  
-> (18, 'Ediciones Paidós', 'España', 1945),  
-> (19, 'Editorial Minotauro', 'España', 1955),  
-> (20, 'Editorial Ariel', 'España', 1942);  
Query OK, 20 rows affected (0.003 sec)  
Records: 20 Duplicates: 0 Warnings: 0  
  
MariaDB [Libros]> INSERT INTO Autor VALUES  
(6, 'Carlos Fuentes', 'México', '1928-11-11'),  
-> (1, 'Gabriel García Márquez', 'Colombia', '1927-03-06'),  
-> (2, 'Isabel Allende', 'Chile', '1942-08-02'),  
-> (3, 'Mario Vargas Llosa', 'Perú', '1936-03-28'),  
-> (4, 'Julio Cortázar', 'Argentina', '1914-08-26'),  
-> (5, 'Jorge Luis Borges', 'Argentina', '1899-08-24'),  
-> (6, 'Carlos Fuentes', 'México', '1928-11-11'),  
-> (7, 'Octavio Paz', 'México', '1914-03-31'),  
-> (8, 'Pablo Neruda', 'Chile', '1904-07-12'),
```



```
root@maquina01:~#
-> (9, 'Fondo de Cultura Económica', 'México', 1934),
-> (10, 'Anagrama', 'España', 1969),
-> (11, 'Editorial Norma', 'Colombia', 1960),
-> (12, 'Editorial Sudamericana', 'Argentina', 1939),
-> (13, 'Santillana', 'España', 1960),
-> (14, 'Siglo XXI Editores', 'México', 1965),
-> (15, 'Seix Barral', 'España', 1911),
-> (16, 'Editorial Porrúa', 'México', 1900),
-> (17, 'Editorial Trillas', 'México', 1959),
-> (18, 'Ediciones Paidós', 'España', 1945),
-> (19, 'Editorial Minotauro', 'España', 1955),
-> (20, 'Editorial Ariel', 'España', 1942);
Query OK, 20 rows affected (0.003 sec)
Records: 20 Duplicates: 0 Warnings: 0

MariaDB [Libros]> INSERT INTO Autor VALUES
(6, 'Carlos Fuentes', 'México', '1928-11-11'),
-> (1, 'Gabriel García Márquez', 'Colombia', '1927-03-06'),
-> (2, 'Isabel Allende', 'Chile', '1942-08-02'),
-> (3, 'Mario Vargas Llosa', 'Perú', '1936-03-28'),
-> (4, 'Julio Cortázar', 'Argentina', '1914-08-26'),
-> (5, 'Jorge Luis Borges', 'Argentina', '1899-08-24'),
-> (6, 'Carlos Fuentes', 'México', '1928-11-11'),
-> (7, 'Octavio Paz', 'México', '1914-03-31'),
-> (8, 'Pablo Neruda', 'Chile', '1904-07-12'),
-> (9, 'Laura Esquivel', 'México', '1950-09-30'),
-> (10, 'Juan Rulfo', 'México', '1917-05-16'),
-> (11, 'Miguel de Cervantes', 'España', '1547-09-29'),
-> (12, 'Federico García Lorca', 'España', '1898-06-05'),
-> (13, 'Camillo José Cela', 'España', '1916-05-11'),
-> (14, 'J.K. Rowling', 'Reino Unido', '1965-07-31'),
-> (15, 'George R.R. Martin', 'Estados Unidos', '1948-09-20'),
-> (16, 'Stephen King', 'Estados Unidos', '1947-09-21'),
-> (17, 'Haruki Murakami', 'Japón', '1949-01-12'),
-> (18, 'Banana Yoshimoto', 'Japón', '1964-07-24'),
-> (19, 'Umberto Eco', 'Italia', '1932-01-05'),
-> (20, 'Italo Calvino', 'Italia', '1923-10-15');
Query OK, 20 rows affected (0.003 sec)
Records: 20 Duplicates: 0 Warnings: 0

MariaDB [Libros]> INSERT INTO Categoria VALUES
-> (1, 'Novela', 'Narración extensa en prosa'),
-> (2, 'Cuento', 'Narración breve en prosa'),
-> (3, 'Poesía', 'Obra literaria en verso'),
-> (4, 'Ensayo', 'Texto argumentativo sobre un tema'),
-> (5, 'Teatro', 'Obra dramática'),
-> (6, 'Historia', 'Obras históricas'),
-> (7, 'Biografía', 'Narración de vida de una persona'),
-> (8, 'Ciencia Ficción', 'Narración futurista o tecnológica'),
```

```
root@maquina01:~#
-> (8, 'Pablo Neruda', 'Chile', '1904-07-12'),
-> (9, 'Laura Esquivel', 'México', '1950-09-30'),
-> (10, 'Juan Rulfo', 'México', '1917-05-16'),
-> (11, 'Miguel de Cervantes', 'España', '1547-09-29'),
-> (12, 'Federico García Lorca', 'España', '1898-06-05'),
-> (13, 'Camillo José Cela', 'España', '1916-05-11'),
-> (14, 'J.K. Rowling', 'Reino Unido', '1965-07-31'),
-> (15, 'George R.R. Martin', 'Estados Unidos', '1948-09-20'),
-> (16, 'Stephen King', 'Estados Unidos', '1947-09-21'),
-> (17, 'Haruki Murakami', 'Japón', '1949-01-12'),
-> (18, 'Banana Yoshimoto', 'Japón', '1964-07-24'),
-> (19, 'Umberto Eco', 'Italia', '1932-01-05'),
-> (20, 'Italo Calvino', 'Italia', '1923-10-15');
Query OK, 20 rows affected (0.003 sec)
Records: 20 Duplicates: 0 Warnings: 0

MariaDB [Libros]> INSERT INTO Categoria VALUES
-> (1, 'Novela', 'Narración extensa en prosa'),
-> (2, 'Cuento', 'Narración breve en prosa'),
-> (3, 'Poesía', 'Obra literaria en verso'),
-> (4, 'Ensayo', 'Texto argumentativo sobre un tema'),
-> (5, 'Teatro', 'Obra dramática'),
-> (6, 'Historia', 'Obras históricas'),
-> (7, 'Biografía', 'Narración de vida de una persona'),
-> (8, 'Ciencia Ficción', 'Narración futurista o tecnológica'),
-> (9, 'Fantasía', 'Narración de mundos imaginarios'),
-> (10, 'Misterio', 'Obras de intriga o suspense'),
-> (11, 'Romance', 'Narraciones amorosas'),
-> (12, 'Aventura', 'Obras con viajes y hazañas'),
-> (13, 'Terror', 'Obras con elementos de horror'),
-> (14, 'Crónica', 'Relatos periodísticos o históricos'),
-> (15, 'Filosofía', 'Obras de pensamiento crítico'),
-> (16, 'Política', 'Textos relacionados con política'),
-> (17, 'Psicología', 'Obras de análisis de la mente'),
-> (18, 'Educación', 'Textos pedagógicos'),
-> (19, 'Economía', 'Obras sobre economía'),
-> (20, 'Arte', 'Obras relacionadas con artes visuales o escénicas');
Query OK, 20 rows affected (0.003 sec)
Records: 20 Duplicates: 0 Warnings: 0

MariaDB [Libros]> INSERT INTO Libro (isbn, titulo, id_autor, id_categoria, id_editorial, anio_publicacion, price, stock, formato) VALUES
-> ('978-0-123456-01-1', 'Introducción a la Inteligencia Artificial', 1, 1, 1, 2021, 35.50, 15, 'Tapa dura'),
-> ('978-0-123456-01-2', 'Historia Universal', 2, 2, 2, 2019, 28.99, 8, 'Tapa blanda'),
-> ('978-0-123456-01-3', 'El Viaje del Programador', 3, 1, 3, 2020, 42.00, 12, 'Digital'),
-> ('978-0-123456-01-4', 'Cocina Mediterránea', 4, 3, 4, 2018, 22.50, 20, 'T
```

```
root@maquina01:~#
-> ('978-0-123456-01-4', 'Cocina Mediterránea', 4, 3, 4, 2018, 22.50, 20, 'Tapa blanda'),
-> ('978-0-123456-01-5', 'Programación en C++', 3, 1, 5, 2022, 55.00, 10, 'Tapa dura'),
-> ('978-0-123456-01-6', 'La Segunda Guerra Mundial', 2, 2, 6, 2017, 30.00, 7, 'Tapa dura'),
-> ('978-0-123456-01-7', 'Diseño UX/UI', 5, 1, 7, 2023, 48.75, 18, 'Digital'),
-> ('978-0-123456-01-8', 'Matemáticas Aplicadas', 6, 4, 8, 2016, 27.80, 6, 'Tapa dura'),
-> ('978-0-123456-01-9', 'El Poder de la Mente', 7, 5, 9, 2015, 19.90, 14, 'Tapa blanda'),
-> ('978-0-123456-02-0', 'Redes de Computadoras', 3, 1, 1, 2020, 60.00, 9, 'Tapa dura'),
-> ('978-0-123456-02-1', 'Literatura Clásica', 8, 2, 2, 2014, 15.00, 25, 'Tapa blanda'),
-> ('978-0-123456-02-2', 'Física Moderna', 9, 4, 3, 2021, 45.20, 11, 'Digital'),
-> ('978-0-123456-02-3', 'Arte Contemporáneo', 10, 6, 4, 2019, 33.10, 5, 'Tapa dura'),
-> ('978-0-123456-02-4', 'Marketing Digital', 11, 7, 5, 2022, 39.95, 13, 'Digital'),
-> ('978-0-123456-02-5', 'Psicología Positiva', 12, 5, 6, 2018, 25.60, 16, 'Tapa blanda'),
-> ('978-0-123456-02-6', 'Economía Global', 13, 9, 7, 2020, 41.70, 10, 'Tapa dura'),
-> ('978-0-123456-02-7', 'Java para Principiantes', 3, 1, 8, 2023, 50.00, 22, 'Digital'),
-> ('978-0-123456-02-8', 'Ecología y Medio Ambiente', 14, 4, 9, 2017, 29.40, 7, 'Tapa dura'),
-> ('978-0-123456-02-9', 'Poesía Latinoamericana', 15, 3, 1, 2016, 18.00, 30, 'Tapa blanda'),
-> ('978-0-123456-03-0', 'Big Data y Analítica', 5, 1, 2, 2021, 65.00, 12, 'Digital');
Query OK, 20 rows affected (0.006 sec)
Records: 20 Duplicates: 0 Warnings: 0

MariaDB [Libros]> SHOW TABLES;
+-----+
| Tables_in_Libros |
+-----+
| Autor            |
| Categoria        |
| Editorial        |
| Libro            |
+-----+
4 rows in set (0.001 sec)

MariaDB [Libros]>
```

Luego en otra VM se creó el directorio de *Microservices* donde se instalaron las *herramientas de desarrollador*, la *base de datos de MariaDB* de manera segura, el lenguaje de programación *C* y *C++*, así como paquete *python-devel*, que contiene los archivos de encabezado y la información necesaria para compilar módulos de extensión de Python y el paquete *flask-mysqldb* en Python, que es un conector para bases de datos MySQL que funciona con el microframework Flask. Esto permite conectar una aplicación web desarrollada con Flask a una base de datos MySQL y realizar operaciones como guardar, leer, actualizar y eliminar datos usando consultas SQL. Por último, se creó que usuario *libro_user* con el password *666*, que más adelante se usará para el microservicio.

```
m03681557@maquina01:~/Microservices
# Using username "m03681557".
# Authenticating with public key "MAGGIE\mocco@Maggie"
Last login: Mon Sep 1 16:10:19 2025 from 201.163.86.212
[m03681557@maquina01 ~]$ FWD
-bash: FWD: command not found
[m03681557@maquina01 ~]$ pwd
/home/m03681557
[m03681557@maquina01 ~]$ mkdir Microservices
[m03681557@maquina01 ~]$ cd Microservices/
[m03681557@maquina01 Microservices]$ python --version
Python 3.9.23
[m03681557@maquina01 Microservices]$ sudo dnf groupinstall "Development Tools" -y

Complete!
[m03681557@maquina01 Microservices]$ sudo dnf install mariadb* -y

Complete!
[m03681557@maquina01 Microservices]$ sudo dnf install gcc* -y
Last metadata expiration check: 0:15:30 ago on Mon 01 Sep 2025 04:39:28 PM UTC.

[m03681557@maquina01 Microservices]$ sudo dnf install python-devel
Last metadata expiration check: 0:15:30 ago on Mon 01 Sep 2025 04:39:28 PM UTC.
```

```
Complete!
[m03681557@maquina01 Microservices]$ pip install flask-mysqldb
Defaulting to user installation because normal site-packages is not writeable
Collecting flask-mysqldb
```

```
MariaDB [(none)]> SET PASSWORD FOR 'libro_user'@'localhost' =
PASSWORD('666');
Query OK, 0 rows affected (0.003 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON Libros.* TO 'libro_
user'@'localhost';
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> exit;
Bye
[m03681557@maquina01 ~]$ mysql -u libro_user -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 21
Server version: 10.5.27-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and
others.

Type 'help;' or 'h' for help. Type 'c' to clear the current
input statement.

MariaDB [(none)]>
```

Después de eso, dentro del directorio de *Microservices* se creó el subdirectorio *micro01*. Dentro de este subdirectorio se creó previamente un microservicio que usando la base de datos Libros, el usuario puede comunicarse con XML a esta a través de endpoints predeterminados. El prompt usado fue el siguiente:

Crear un microservicio con los siguientes endpoints utilizando la base de datos llamada Libros, con el usuario libro_user y password 666. La respuesta de cada microservicio deberá ser XML. Utiliza las siguientes librerías xml.etree, requests, mysql EXCLUSIVAMENTE.

/api/books ← ver todos los libros

/api/books/ISBN ← buscar por # de ISBN

/api/books/formats/ ← buscar por formato

/api/books/author/ ← buscar por nombre de autor

/api/books/book/insert ← insertar libro

/api/books/book/delete ← borrar libro por ISBN

/api/books/book/update ← actualizar libro por ISBN

Asimismo, usa el siguiente archivo XML para todos los endpoints:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<catalog>
```

```
  <book isbn="978-0-123456-01-1">
```

```
    <title>Introducción a la Inteligencia Artificial</title>
```

```
    <author>María López</author>
```

```
<year>2021</year>

<genre>Tecnología</genre>

<price>35.50</price>

<stock>15</stock>

<format>Tapa dura</format>

</book>
```

```
<book isbn="978-0-123456-01-2">

  <title>Historia Universal</title>

  <author>Carlos Martínez</author>

  <year>2019</year>

  <genre>Historia</genre>

  <price>28.99</price>

  <stock>8</stock>

  <format>Tapa blanda</format>

</book>
```

```
<book isbn="978-0-123456-01-3">

  <title>El Viaje del Programador</title>

  <author>Ana Torres</author>

  <year>2020</year>

  <genre>Computación</genre>

  <price>42.00</price>

  <stock>12</stock>

  <format>Digital</format>

</book>
```

```
<book isbn="978-0-123456-01-4">  
  <title>Cocina Mediterránea</title>  
  <author>Laura Fernández</author>  
  <year>2018</year>  
  <genre>Gastronomía</genre>  
  <price>22.50</price>  
  <stock>20</stock>  
  <format>Tapa blanda</format>  
</book>
```

```
<book isbn="978-0-123456-01-5">  
  <title>Programación en C++</title>  
  <author>José Ramírez</author>  
  <year>2022</year>  
  <genre>Computación</genre>  
  <price>55.00</price>  
  <stock>10</stock>  
  <format>Tapa dura</format>  
</book>
```

```
<book isbn="978-0-123456-01-6">  
  <title>La Segunda Guerra Mundial</title>  
  <author>Pedro Sánchez</author>  
  <year>2017</year>  
  <genre>Historia</genre>
```

```
<price>30.00</price>

<stock>7</stock>

<format>Tapa dura</format>

</book>
```

```
<book isbn="978-0-123456-01-7">

  <title>Diseño UX/UI</title>

  <author>Sofía Gómez</author>

  <year>2023</year>

  <genre>Tecnología</genre>

  <price>48.75</price>

  <stock>18</stock>

  <format>Digital</format>

</book>
```

```
<book isbn="978-0-123456-01-8">

  <title>Matemáticas Aplicadas</title>

  <author>Andrés Castillo</author>

  <year>2016</year>

  <genre>Educación</genre>

  <price>27.80</price>

  <stock>6</stock>

  <format>Tapa dura</format>

</book>
```

```
<book isbn="978-0-123456-01-9">
```

```
<title>El Poder de la Mente</title>
<author>Lucía Morales</author>
<year>2015</year>
<genre>Autoayuda</genre>
<price>19.90</price>
<stock>14</stock>
<format>Tapa blanda</format>
</book>
```

```
<book isbn="978-0-123456-02-0">
  <title>Redes de Computadoras</title>
  <author>Roberto Díaz</author>
  <year>2020</year>
  <genre>Computación</genre>
  <price>60.00</price>
  <stock>9</stock>
  <format>Tapa dura</format>
</book>
```

```
<book isbn="978-0-123456-02-1">
  <title>Literatura Clásica</title>
  <author>Isabel Herrera</author>
  <year>2014</year>
  <genre>Literatura</genre>
  <price>15.00</price>
  <stock>25</stock>
```

<format>Tapa blanda</format>
</book>

<book isbn="978-0-123456-02-2">
 <title>Física Moderna</title>
 <author>Miguel Ángel Ruiz</author>
 <year>2021</year>
 <genre>Ciencia</genre>
 <price>45.20</price>
 <stock>11</stock>
 <format>Digital</format>
</book>

<book isbn="978-0-123456-02-3">
 <title>Arte Contemporáneo</title>
 <author>Patricia Vargas</author>
 <year>2019</year>
 <genre>Arte</genre>
 <price>33.10</price>
 <stock>5</stock>
 <format>Tapa dura</format>
</book>

<book isbn="978-0-123456-02-4">
 <title>Marketing Digital</title>
 <author>Diego Ramos</author>

<year>2022</year>
<genre>Negocios</genre>
<price>39.95</price>
<stock>13</stock>
<format>Digital</format>
</book>

<book isbn="978-0-123456-02-5">
 <title>Psicología Positiva</title>
 <author>Carolina Ortiz</author>
 <year>2018</year>
 <genre>Psicología</genre>
 <price>25.60</price>
 <stock>16</stock>
 <format>Tapa blanda</format>
</book>

<book isbn="978-0-123456-02-6">
 <title>Economía Global</title>
 <author>Fernando Gutiérrez</author>
 <year>2020</year>
 <genre>Economía</genre>
 <price>41.70</price>
 <stock>10</stock>
 <format>Tapa dura</format>
</book>

```
<book isbn="978-0-123456-02-7">  
  <title>Java para Principiantes</title>  
  <author>Rosa Jiménez</author>  
  <year>2023</year>  
  <genre>Computación</genre>  
  <price>50.00</price>  
  <stock>22</stock>  
  <format>Digital</format>  
</book>
```

```
<book isbn="978-0-123456-02-8">  
  <title>Ecología y Medio Ambiente</title>  
  <author>Javier López</author>  
  <year>2017</year>  
  <genre>Ciencia</genre>  
  <price>29.40</price>  
  <stock>7</stock>  
  <format>Tapa dura</format>  
</book>
```

```
<book isbn="978-0-123456-02-9">  
  <title>Poesía Latinoamericana</title>  
  <author>Elena Castro</author>  
  <year>2016</year>  
  <genre>Poesía</genre>
```

```

    <price>18.00</price>

    <stock>30</stock>

    <format>Tapa blanda</format>
</book>

<book isbn="978-0-123456-03-0">

    <title>Big Data y Analítica</title>

    <author>Hugo Medina</author>

    <author>Mariana Pérez</author>

    <year>2021</year>

    <genre>Tecnología</genre>

    <price>65.00</price>

    <stock>12</stock>

    <format>Digital</format>

</book>

</catalog>

```

Al ejecutarlo, la IA nos dio el contenido del archivo ejecutable **main.py**, el cual contiene el siguiente código:

```

Python
from flask import Flask, request, Response
import MySQLdb
import xml.etree.ElementTree as ET

app = Flask(__name__)

# -----
# Conexión a la base de datos
# -----
def get_db_connection():
    return MySQLdb.connect(
        host="localhost",
        user="libro_user",
        passwd="666",

```

```

        db="Libros",
        charset='utf8'
    )

# -----
# Función para convertir resultados a XML
# -----
def books_to_xml(books):
    catalog = ET.Element('catalog')
    for book in books:
        book_elem = ET.Element('book', isbn=book.get('isbn', ''))
        # Título
        title = ET.SubElement(book_elem, 'title')
        title.text = book.get('titulo', 'Desconocido')
        # Autores
        authors_field = book.get('authors', '')
        authors = authors_field.split(',') if authors_field else ['Desconocido']
        for author_name in authors:
            author = ET.SubElement(book_elem, 'author')
            author.text = author_name.strip()
        # Año
        year = ET.SubElement(book_elem, 'year')
        year.text = str(book.get('anio_publicacion', '0'))
        # Género
        genre = ET.SubElement(book_elem, 'genre')
        genre.text = book.get('genre_name', 'Desconocido')
        # Precio
        price = ET.SubElement(book_elem, 'price')
        price.text = str(book.get('price', '0.0'))
        # Stock
        stock = ET.SubElement(book_elem, 'stock')
        stock.text = str(book.get('stock', '0'))
        # Formato
        fmt = ET.SubElement(book_elem, 'format')
        fmt.text = book.get('formato', 'Desconocido')
        catalog.append(book_elem)
    xml_str = ET.tostring(catalog, encoding='utf-8', xml_declaration=True)
    return xml_str

# -----
# Endpoints
# -----

@app.route('/api/books', methods=['GET'])
def get_books():
    try:
        conn = get_db_connection()
        cursor = conn.cursor(MySQLdb.cursors.DictCursor)
        cursor.execute("""
            SELECT l.*,
                GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                c.nombre AS genre_name
        """)
    
```

```

        FROM Libro l
        LEFT JOIN Autor a ON l.id_autor = a.id_autor
        LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
        GROUP BY l.isbn
    """
    books = cursor.fetchall()
    conn.close()
    xml = books_to_xml(books)
    return Response(xml, mimetype='application/xml')
except Exception as e:
    return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/<isbn>', methods=['GET'])
def get_book_by_isbn(isbn):
    try:
        conn = get_db_connection()
        cursor = conn.cursor(MySQLdb.cursors.DictCursor)
        cursor.execute("""
            SELECT l.*,
                GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            WHERE l.isbn=%s
            GROUP BY l.isbn
        """, (isbn,))
        book = cursor.fetchone()
        conn.close()
        if book:
            xml = books_to_xml([book])
            return Response(xml, mimetype='application/xml')
        else:
            return Response("<?xml version='1.0' encoding='UTF-8'?><error>Libro no
encontrado</error>", mimetype='application/xml', status=404)
    except Exception as e:
        return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/formats/<formato>', methods=['GET'])
def get_books_by_format(formato):
    try:
        conn = get_db_connection()
        cursor = conn.cursor(MySQLdb.cursors.DictCursor)
        cursor.execute("""
            SELECT l.*,
                GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria

```

```

        WHERE l.formato=%s
        GROUP BY l.isbn
    """ , (formato,))
    books = cursor.fetchall()
    conn.close()
    xml = books_to_xml(books)
    return Response(xml, mimetype='application/xml')
except Exception as e:
    return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/author/<author_name>', methods=['GET'])
def get_books_by_author(author_name):
    try:
        conn = get_db_connection()
        cursor = conn.cursor(MySQLdb.cursors.DictCursor)
        cursor.execute("""
            SELECT l.*,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            WHERE a.nombre LIKE %s
            GROUP BY l.isbn
        """, ('%' + author_name + '%',))
        books = cursor.fetchall()
        conn.close()
        xml = books_to_xml(books)
        return Response(xml, mimetype='application/xml')
    except Exception as e:
        return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/book/insert', methods=['POST'])
def insert_book():
    try:
        data = request.json
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO Libro (isbn, titulo, id_autor, id_categoria, id_editorial,
            anio_publicacion, price, stock, formato)
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)
        """, (
            data.get('isbn'), data.get('titulo'), data.get('id_autor'),
            data.get('id_categoria'), data.get('id_editorial'),
            data.get('anio_publicacion'), data.get('price'), data.get('stock'),
            data.get('formato')
        ))
        conn.commit()
        conn.close()

```

```

        return Response("<?xml version='1.0' encoding='UTF-8'?><success>Libro
insertado</success>", mimetype='application/xml')
    except Exception as e:
        return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/book/delete', methods=['DELETE'])
def delete_book():
    try:
        isbn = request.args.get('isbn')
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("DELETE FROM Libro WHERE isbn=%s", (isbn,))
        conn.commit()
        conn.close()
        return Response("<?xml version='1.0' encoding='UTF-8'?><success>Libro
eliminado</success>", mimetype='application/xml')
    except Exception as e:
        return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/book/update', methods=['PUT'])
def update_book():
    try:
        data = request.json
        isbn = data.get('isbn')
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("""
            UPDATE Libro SET
            titulo=%s,
            id_autor=%s,
            id_categoria=%s,
            id_editorial=%s,
            anio_publicacion=%s,
            price=%s,
            stock=%s,
            formato=%s
            WHERE isbn=%s
            """, (
                data.get('titulo'), data.get('id_autor'), data.get('id_categoria'),
data.get('id_editorial'),
                data.get('anio_publicacion'), data.get('price'), data.get('stock'),
data.get('formato'), isbn
            ))
        conn.commit()
        conn.close()
        return Response("<?xml version='1.0' encoding='UTF-8'?><success>Libro
actualizado</success>", mimetype='application/xml')
    except Exception as e:
        return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

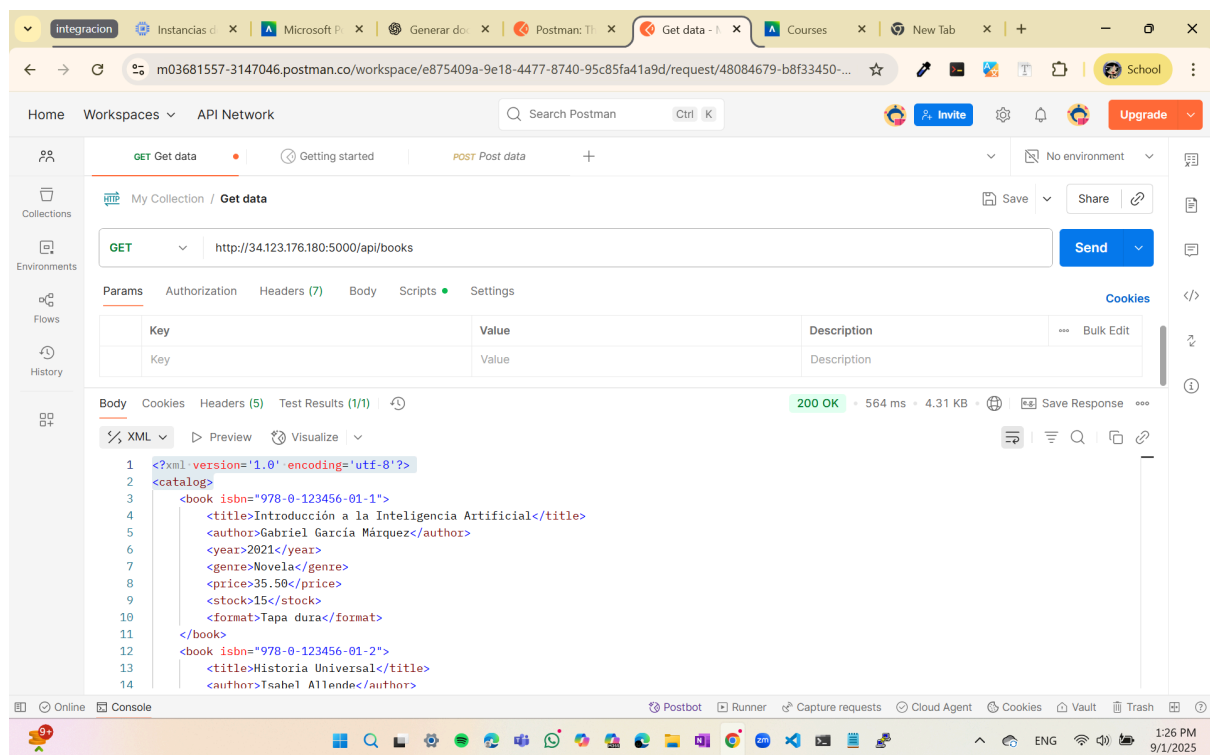
```

```
# -----
if __name__ == '__main__':
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Se copió dicho código y con el comando `sudo vi main.py` se creó e insertó el código de arriba. Para poder visualizar el contenido de este en el navegador, se tuvo que crear una regla de firewall que permitiera acceder a cualquier IP de la red.

```
C:\>gcloud compute firewall-rules create api-flask --network=default --direction=INGRESS --priority=1000 --action=ALLOW --rules=tcp:5000-5005 --source-ranges=0.0.0.0/0
Creating firewall...Created [https://www.googleapis.com/compute/v1/projects/integracion-468300/global/firewalls/api-flask].
Creating firewall...done.
NAME          NETWORK  DIRECTION  PRIORITY  ALLOW          DENY  DISABLED
api-flask     default  INGRESS    1000      tcp:5000-5005  False
C:\>|
```

Al tener configurada correctamente el firewall, se copió la IP de la máquina virtual y se agregó la ruta del archivo con `/api/books` para probar dicha URL con POSTMAN. Este es el resultado visible:



Como se puede observar, ya se tiene la base del microservicio existente, no obstante **este aún no integra JWT para la autenticación de los usuarios**. Por ello se harán

modificaciones al main.py existente, con ayuda de la IA, para que ahora los usuarios que quieran acceder a la información que contiene la BD Libros puedan hacerlo después de autenticarse.

Lo primero que hizo fue brindarle a la IA el dump de la BD y el ejecutable main.py para que me ayudara a editar este microservicio con el siguiente prompt:

Con los archivos proporcionados, genera un microservicio, o en su defecto adapta el microservicio existente, en Flask que utilice JWT para autenticar usuarios contra la base de datos llamada Libros. El microservicio deberá permitir registrar usuarios, iniciar sesión, generar tokens JWT así como refrescarlos y proteger rutas que solo se puedan acceder con un token válido. Como te comente, esta base de datos de la librería ya existe más consume XML. No quiero que conviertas estos XML sino que a partir de estos uses JSON para la autenticación. Deberá verificar o probar que esto funcione con POSTMAN. No olvides manejar de manera correcta el mecanismo de seguridad CORS.

La respuesta de la IA fue la siguiente:

Esta nueva versión de tu archivo **main.py** incluye:

- Una nueva tabla de usuarios en tu base de datos para almacenar las credenciales de los usuarios.
- Lógica de autenticación de usuarios utilizando Flask-JWT-Extended y passlib para un hash seguro de contraseñas.
- Importamos la biblioteca 'from flask_cors import CORS' para evitar problemas con CORS cuando su aplicación web intente comunicarse con su microservicio.
- Nuevos puntos finales de API para tokens /register, /login y /refresh.
- Puntos finales protegidos que requieren un JWT válido para acceder.
- No olvide crear la tabla usuarios en su BD e instalar las bibliotecas necesarias.

```
Python
from flask import Flask, request, Response, jsonify
import MySQLdb
import xml.etree.ElementTree as ET
from datetime import timedelta
from flask_jwt_extended import (
    JWTManager, create_access_token, create_refresh_token,
    jwt_required, get_jwt_identity
)
from passlib.hash import pbkdf2_sha256
from flask_cors import CORS # 1. Importa CORS

# ----- Flask & JWT Configuration -----
app = Flask(__name__)
CORS(app) # 2. Habilita CORS para toda la aplicación

# IMPORTANT: In a production environment, use environment variables to store this key.
app.config['JWT_SECRET_KEY'] = 'your-super-secret-key-here'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
```

```

app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(days=30)
jwt = JWTManager(app)

# -----
# Database Connection
# -----
def get_db_connection():
    return MySQLdb.connect(
        host="localhost",
        user="libro_user",
        passwd="666",
        db="Libros",
        charset='utf8'
    )

# -----
# XML Conversion Function
# -----
def books_to_xml(books):
    catalog = ET.Element('catalog')
    for book in books:
        book_elem = ET.Element('book', isbn=book.get('isbn', ''))
        # Título
        title = ET.SubElement(book_elem, 'title')
        title.text = book.get('titulo', 'Desconocido')
        # Autores
        authors_field = book.get('authors', '')
        authors = authors_field.split(',') if authors_field else ['Desconocido']
        for author_name in authors:
            author = ET.SubElement(book_elem, 'author')
            author.text = author_name.strip()
        # Año
        year = ET.SubElement(book_elem, 'year')
        year.text = str(book.get('anio_publicacion', '0'))
        # Género
        genre = ET.SubElement(book_elem, 'genre')
        genre.text = book.get('genre_name', 'Desconocido')
        # Precio
        price = ET.SubElement(book_elem, 'price')
        price.text = str(book.get('price', '0.0'))
        # Stock
        stock = ET.SubElement(book_elem, 'stock')
        stock.text = str(book.get('stock', '0'))
        # Formato
        fmt = ET.SubElement(book_elem, 'format')
        fmt.text = book.get('formato', 'Desconocido')
        catalog.append(book_elem)
    xml_str = ET.tostring(catalog, encoding='utf-8', xml_declaration=True)
    return xml_str

# -----
# New Endpoints for User Authentication

```

```

# -----

@app.route('/register', methods=['POST'])
def register():
    data = request.json
    username = data.get('username')
    password = data.get('password')

    if not username or not password:
        return jsonify({"msg": "Username and password are required"}), 400

    conn = None
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # Check if user already exists
        cursor.execute("SELECT * FROM Users WHERE username = %s", (username,))
        if cursor.fetchone():
            return jsonify({"msg": "User already exists"}), 409

        # Hash password and insert into DB
        password_hash = pbkdf2_sha256.hash(password)
        cursor.execute("INSERT INTO Users (username, password_hash) VALUES (%s, %s)",
            (username, password_hash))
        conn.commit()

        return jsonify({"msg": "User created successfully"}), 201

    except Exception as e:
        return jsonify({"msg": "An error occurred", "error": str(e)}), 500
    finally:
        if conn:
            conn.close()

@app.route('/login', methods=['POST'])
def login():
    data = request.json
    username = data.get('username')
    password = data.get('password')

    if not username or not password:
        return jsonify({"msg": "Username and password are required"}), 400

    conn = None
    try:
        conn = get_db_connection()
        cursor = conn.cursor(MySQLdb.cursors.DictCursor)

        # Retrieve user and check password
        cursor.execute("SELECT password_hash FROM Users WHERE username = %s",
            (username,))

```

```

        user = cursor.fetchone()

        if user and pbkdf2_sha256.verify(password, user['password_hash']):
            access_token = create_access_token(identity=username)
            refresh_token = create_refresh_token(identity=username)
            return jsonify(access_token=access_token, refresh_token=refresh_token),
200
        else:
            return jsonify({"msg": "Bad username or password"}), 401
    except Exception as e:
        return jsonify({"msg": "An error occurred", "error": str(e)}), 500
    finally:
        if conn:
            conn.close()

@app.route('/refresh', methods=['POST'])
@jwt_required(refresh=True)
def refresh():
    current_user = get_jwt_identity()
    new_access_token = create_access_token(identity=current_user)
    return jsonify(access_token=new_access_token), 200

# -----
# Existing Endpoints (now protected)
# -----

@app.route('/api/books', methods=['GET'])
@jwt_required()
def get_books():
    try:
        conn = get_db_connection()
        cursor = conn.cursor(MySQLdb.cursors.DictCursor)
        cursor.execute("""
            SELECT l.*,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            GROUP BY l.isbn
        """)
        books = cursor.fetchall()
        conn.close()
        xml = books_to_xml(books)
        return Response(xml, mimetype='application/xml')
    except Exception as e:
        return Response(f"<?xml version='1.0'
encoding='UTF-8' ?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/<isbn>', methods=['GET'])
@jwt_required()
def get_book_by_isbn(isbn):

```

```

try:
    conn = get_db_connection()
    cursor = conn.cursor(MySQLdb.cursors.DictCursor)
    cursor.execute("""
        SELECT l.*,
               GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
               c.nombre AS genre_name
        FROM Libro l
        LEFT JOIN Autor a ON l.id_autor = a.id_autor
        LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
        WHERE l.isbn=%s
        GROUP BY l.isbn
    """, (isbn,))
    book = cursor.fetchone()
    conn.close()
    if book:
        xml = books_to_xml([book])
        return Response(xml, mimetype='application/xml')
    else:
        return Response("<?xml version='1.0' encoding='UTF-8'?><error>Libro no encontrado</error>", mimetype='application/xml', status=404)
except Exception as e:
    return Response(f"<?xml version='1.0' encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/formats/<formato>', methods=['GET'])
@jwt_required()
def get_books_by_format(formato):
    try:
        conn = get_db_connection()
        cursor = conn.cursor(MySQLdb.cursors.DictCursor)
        cursor.execute("""
            SELECT l.*,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            WHERE l.formato=%s
            GROUP BY l.isbn
        """, (formato,))
        books = cursor.fetchall()
        conn.close()
        xml = books_to_xml(books)
        return Response(xml, mimetype='application/xml')
    except Exception as e:
        return Response(f"<?xml version='1.0' encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/author/<author_name>', methods=['GET'])
@jwt_required()
def get_books_by_author(author_name):

```

```

try:
    conn = get_db_connection()
    cursor = conn.cursor(MySQLdb.cursors.DictCursor)
    cursor.execute("""
        SELECT l.*,
               GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
               c.nombre AS genre_name
        FROM Libro l
        LEFT JOIN Autor a ON l.id_autor = a.id_autor
        LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
        WHERE a.nombre LIKE %s
        GROUP BY l.isbn
    """, ('%' + author_name + '%',))
    books = cursor.fetchall()
    conn.close()
    xml = books_to_xml(books)
    return Response(xml, mimetype='application/xml')
except Exception as e:
    return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/book/insert', methods=['POST'])
@jwt_required()
def insert_book():
    try:
        data = request.json
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO Libro (isbn, titulo, id_autor, id_categoria, id_editorial,
            anio_publicacion, price, stock, formato)
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)
        """, (
            data.get('isbn'), data.get('titulo'), data.get('id_autor'),
            data.get('id_categoria'), data.get('id_editorial'),
            data.get('anio_publicacion'), data.get('price'), data.get('stock'),
            data.get('formato')
        ))
        conn.commit()
        conn.close()
        return Response("<?xml version='1.0' encoding='UTF-8'?><success>Libro
insertado</success>", mimetype='application/xml')
    except Exception as e:
        return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/book/delete', methods=['DELETE'])
@jwt_required()
def delete_book():
    try:
        isbn = request.args.get('isbn')
        conn = get_db_connection()

```

```

        cursor = conn.cursor()
        cursor.execute("DELETE FROM Libro WHERE isbn=%s", (isbn,))
        conn.commit()
        conn.close()
        return Response("<?xml version='1.0' encoding='UTF-8'?><success>Libro
eliminado</success>", mimetype='application/xml')
    except Exception as e:
        return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

@app.route('/api/books/book/update', methods=['PUT'])
@jwt_required()
def update_book():
    try:
        data = request.json
        isbn = data.get('isbn')
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("""
            UPDATE Libro SET
            titulo=%s,
            id_autor=%s,
            id_categoria=%s,
            id_editorial=%s,
            anio_publicacion=%s,
            price=%s,
            stock=%s,
            formato=%s
            WHERE isbn=%s
            """, (
                data.get('titulo'), data.get('id_autor'), data.get('id_categoria'),
data.get('id_editorial'),
                data.get('anio_publicacion'), data.get('price'), data.get('stock'),
data.get('formato'), isbn
            ))
        conn.commit()
        conn.close()
        return Response("<?xml version='1.0' encoding='UTF-8'?><success>Libro
actualizado</success>", mimetype='application/xml')
    except Exception as e:
        return Response(f"<?xml version='1.0'
encoding='UTF-8'?><error>{str(e)}</error>", mimetype='application/xml', status=500)

# -----
if __name__ == '__main__':
    app.run(host="0.0.0.0", port=5000, debug=True)

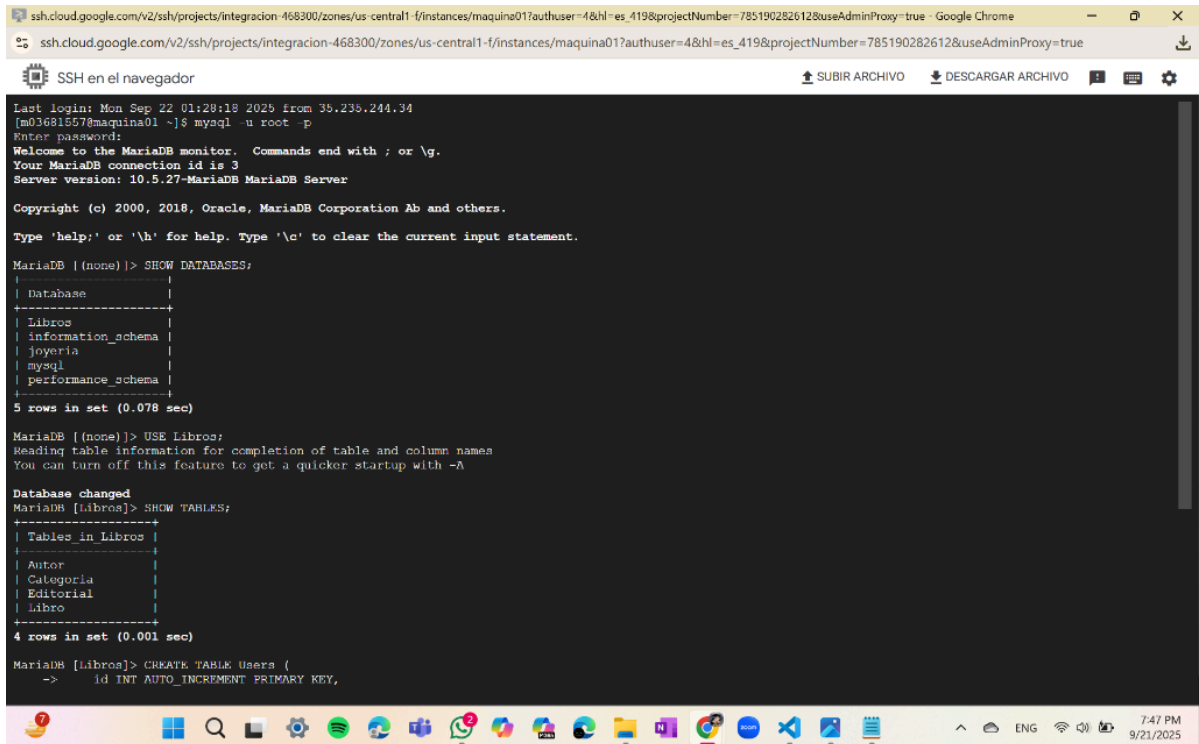
```

Antes de poder correr el microservicio editado, se tuvo que crear la tabla *Users* e instalar JWT para Flask en el directorio *micro01* con el comando **pip install Flask-JWT-Extended passlib** así como la librería de CORS con el comando **pip install Flask-Cors**. El comando SQL para la tabla *Users* es el siguiente:

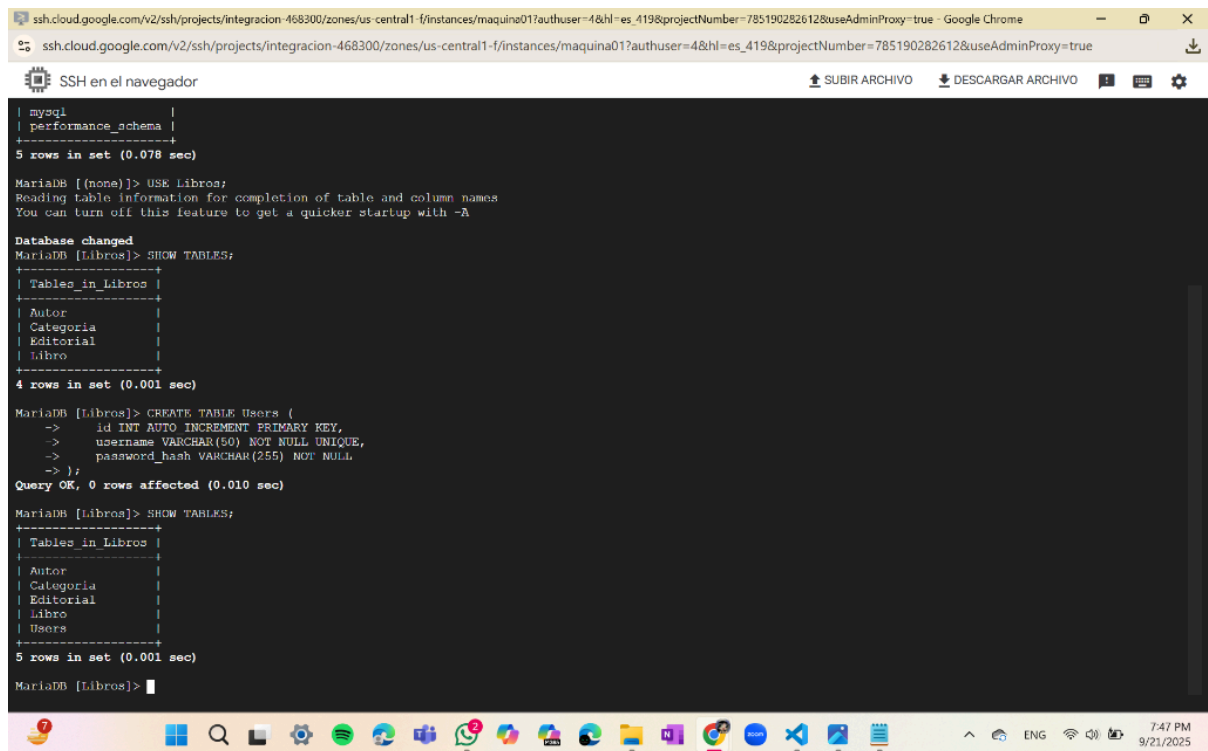
SQL

USE Libros;

```
CREATE TABLE Users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL  
);
```



```
ssh.cloud.google.com/v2/ssh/projects/integracion-468300/zones/us-central1-f/instances/maquina01?authuser=4&hl=es_419&projectNumber=785190282612&useAdminProxy=true - Google Chrome  
ssh.cloud.google.com/v2/ssh/projects/integracion-468300/zones/us-central1-f/instances/maquina01?authuser=4&hl=es_419&projectNumber=785190282612&useAdminProxy=true  
SSH en el navegador  
Last login: Mon Sep 22 01:28:18 2025 from 35.236.244.34  
[m03681557@maquina01 ~]$ mysql -u root -p  
Enter password:  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 3  
Server version: 10.5.27-MariaDB MariaDB Server  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [(none)]> SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| Libros   |  
| information_schema |  
| joyeria  |  
| mysql    |  
| performance_schema |  
+-----+  
5 rows in set (0.078 sec)  
  
MariaDB [(none)]> USE Libros;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
  
Database changed  
MariaDB [Libros]> SHOW TABLES;  
+-----+  
| Tables in Libros |  
+-----+  
| Autor            |  
| Categoria        |  
| Editorial         |  
| Libro            |  
+-----+  
4 rows in set (0.001 sec)  
  
MariaDB [Libros]> CREATE TABLE Users (  
->   id INT AUTO_INCREMENT PRIMARY KEY,
```

```
mysql
performance_schema
5 rows in set (0.078 sec)

MariaDB [(none)]> USE Libros;
Reading Table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [Libros]> SHOW TABLES;
+-----+
| Tables_in_Libros |
+-----+
| Autor             |
| Categoria         |
| Editorial         |
| Libro             |
+-----+
4 rows in set (0.001 sec)

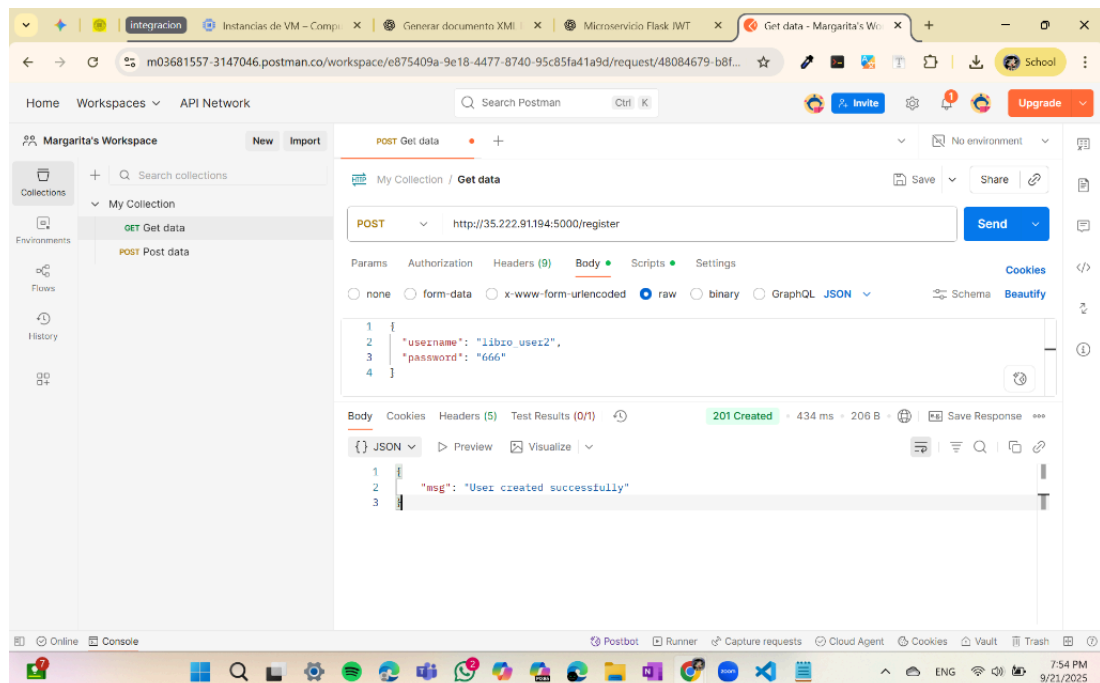
MariaDB [Libros]> CREATE TABLE Users (
  -> id INT AUTO INCREMENT PRIMARY KEY,
  -> username VARCHAR(50) NOT NULL UNIQUE,
  -> password_hash VARCHAR(255) NOT NULL,
  -> );
Query OK, 0 rows affected (0.010 sec)

MariaDB [Libros]> SHOW TABLES;
+-----+
| Tables_in_Libros |
+-----+
| Autor             |
| Categoria         |
| Editorial         |
| Libro             |
| Users            |
+-----+
5 rows in set (0.001 sec)

MariaDB [Libros]>
```

Una vez actualizado el microservicio, se probó que este funcionará con POSTMAN al probar todas las rutas. A continuación se muestra evidencia de algunas de las pruebas:

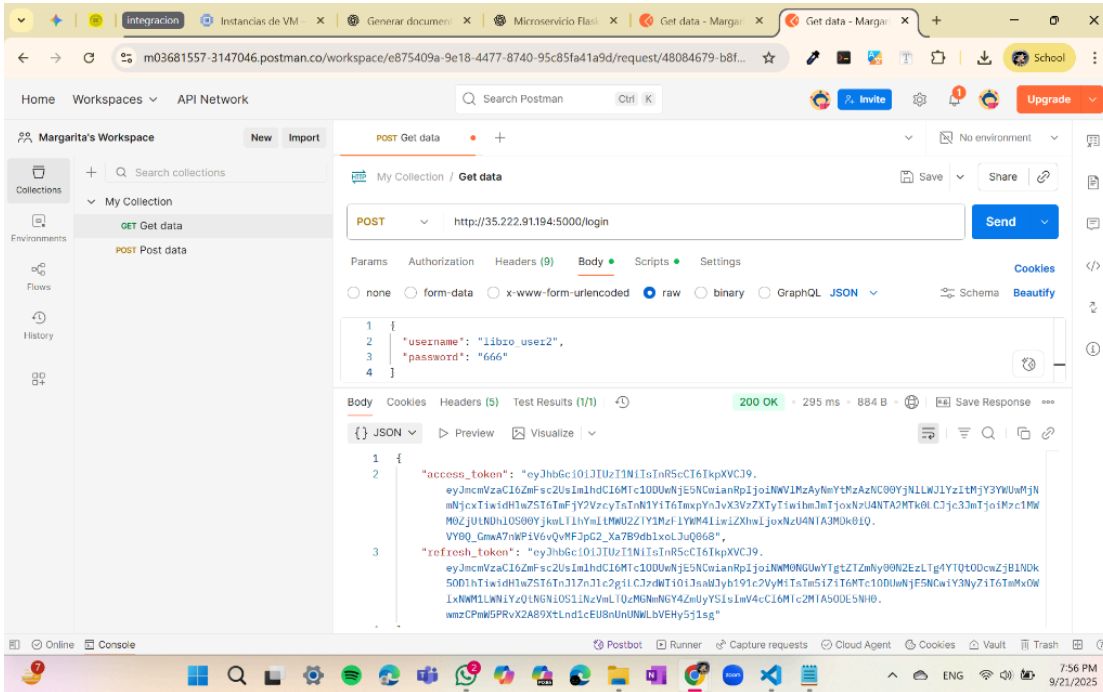
1. registro de usuario



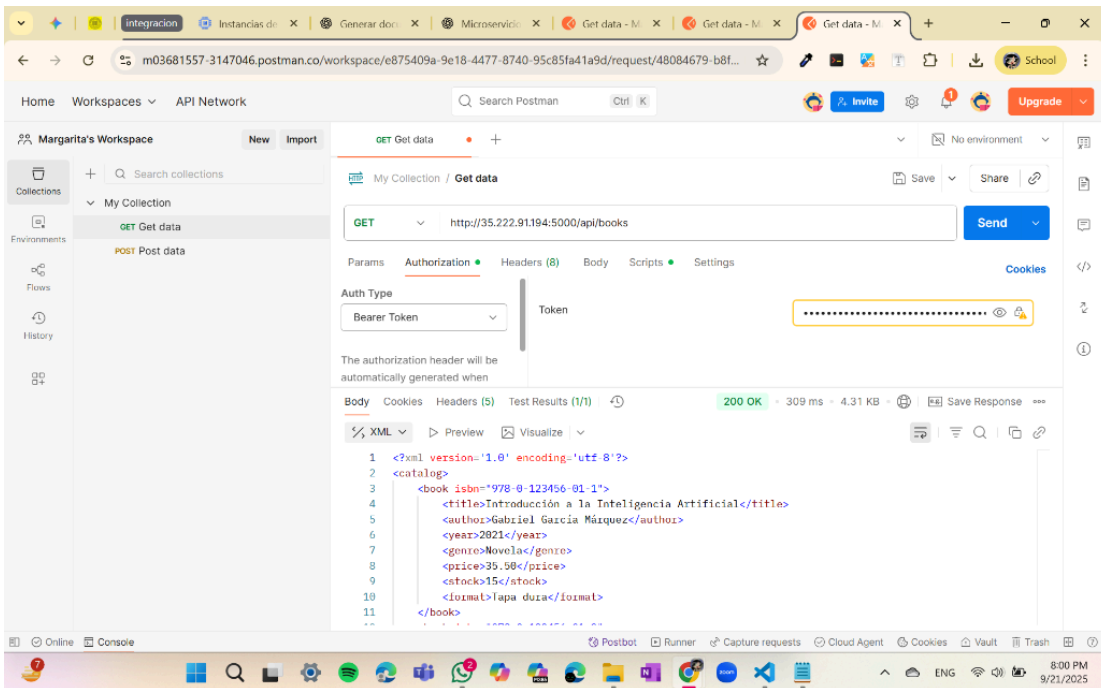
```
MariaDB [libros]> SELECT * FROM Usuario
+-----+-----+-----+
| id | username | password hash |
+-----+-----+-----+
| 1 | libro_user2 | $pbkdf2-sha256$29000$w7jx2lurFYkwIttbe4.xlg$GaeHuJrkjXyhSPdqusMvo609337cfb/skNm90z1lhPgI |
+-----+-----+-----+
1 row in set (0.000 sec)

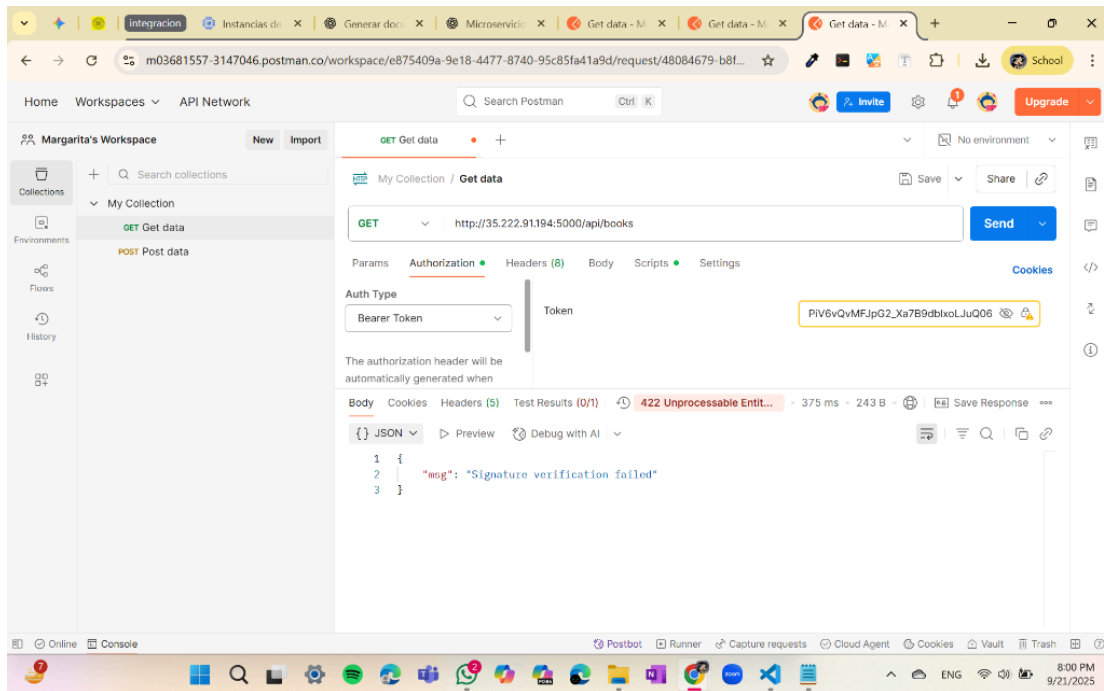
MariaDB [libros]>
```

2. inicio de sesión de usuario para obtener tokens de acceso y refresh

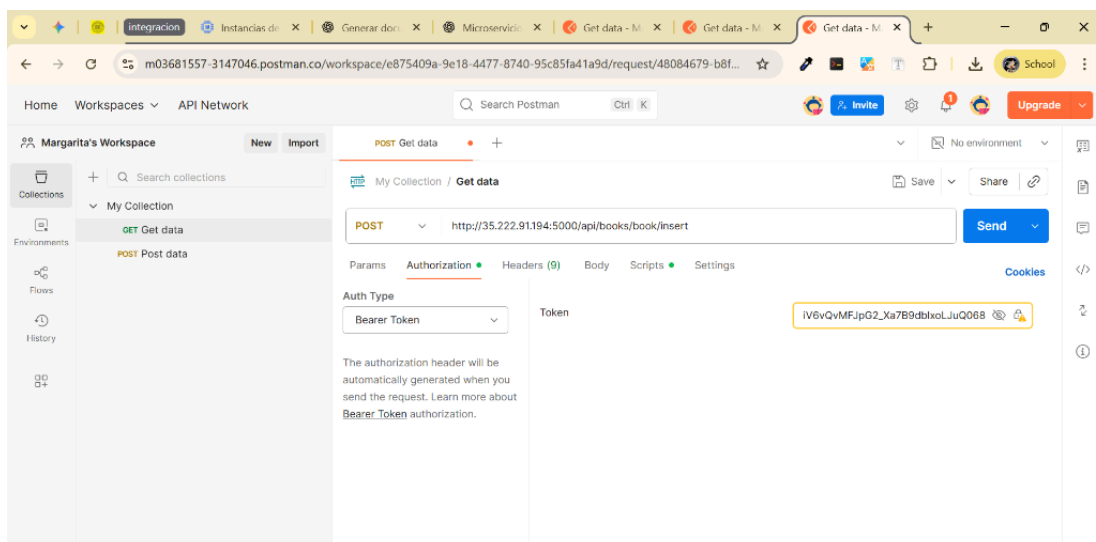


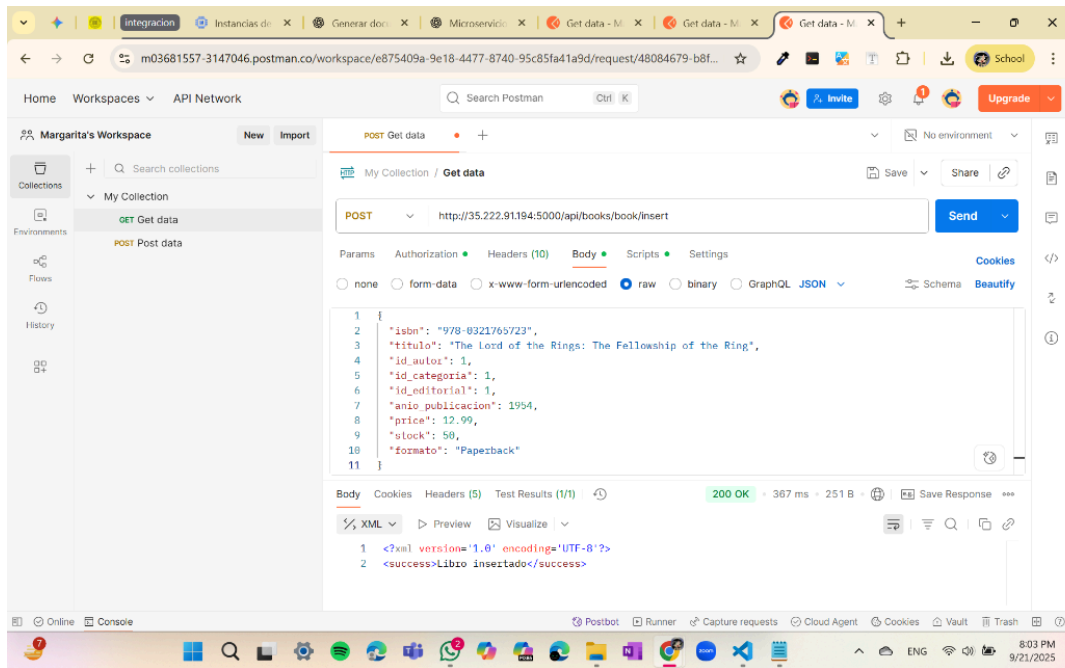
3. ver catálogo de libros al ingresar token (y su respuesta si el token es distinto o inválido)



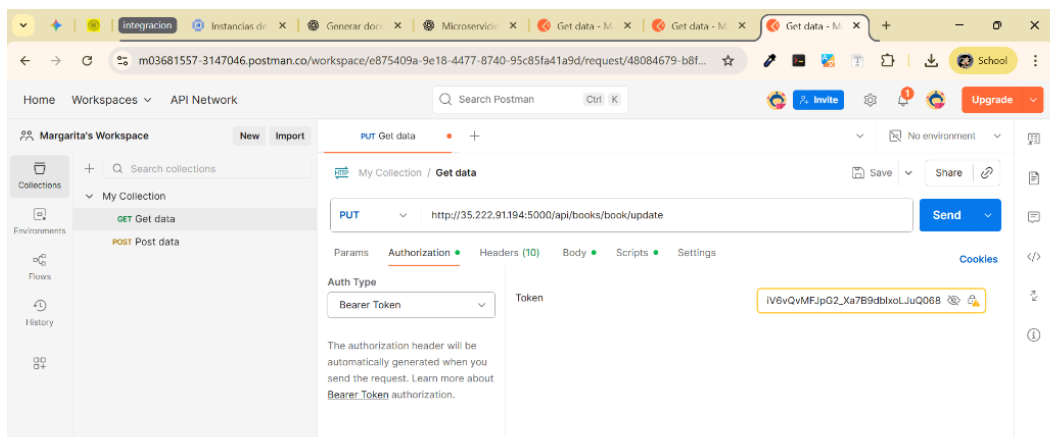


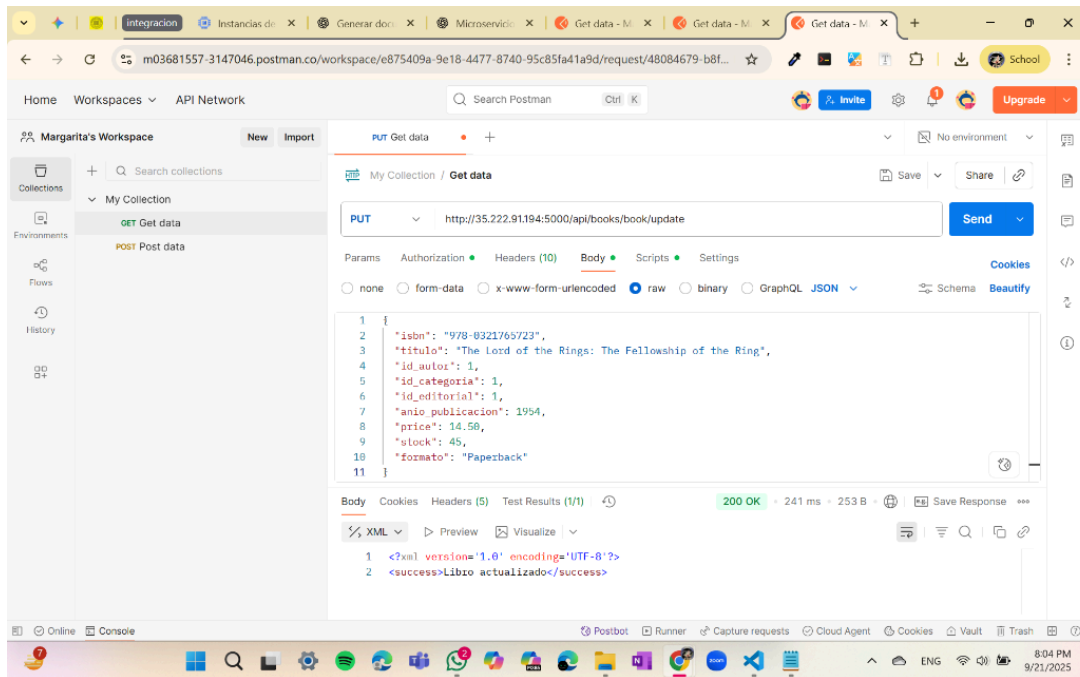
4. insertar libro con el token y los datos del libro en formato JSON



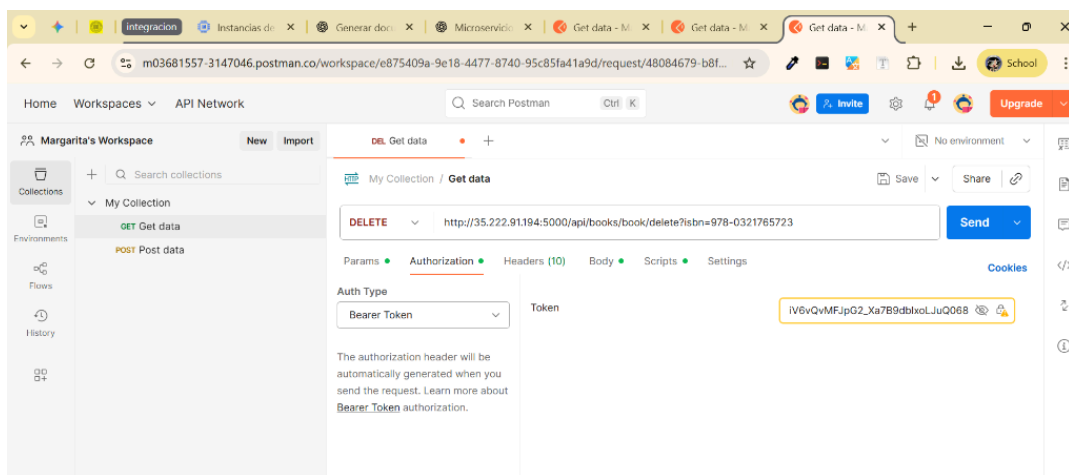


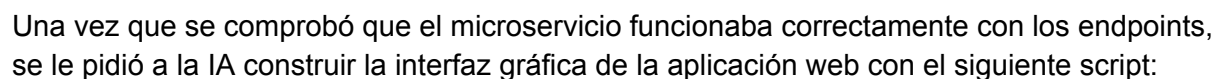
5. actualizar libro con el token y los datos del libro en formato JSON





6. eliminar libro con el token





Además, construye un aplicación web (HTML, CSS, JS) separada que consuma el microservicio creado desde otra máquina, almacenando el token y enviándolo en las cabeceras de las solicitudes. Se deberá incluir logs tanto en el servidor como en el cliente para visualizar cómo funciona el flujo de autenticación con JWT.

En respuesta la IA creo 3 archivos:

1. **index.html:** archivo contiene la estructura de la página web. Tendrá formularios de login, registro, búsqueda, etc. y una sección para mostrar los datos de los libros. También incluye un área de "logs" para que puedas ver lo que está sucediendo en el cliente.
2. **style.css:** archivo que proporciona un estilo básico para hacer la interfaz más atractiva y fácil de usar.
3. **script.js:** este es el archivo más importante. Contiene la lógica para enviar solicitudes, gestionar la autenticación y mostrar los resultados de los logs.

Código index.html:

```
HTML
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cliente de Microservicio de Libros</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Cliente de Microservicio de Libros</h1>

    <div class="auth-section">
      <h2>Autenticación</h2>
      <form id="login-form">
        <input type="text" id="username" placeholder="Usuario" required>
        <input type="password" id="password" placeholder="Contraseña"
required>
        <button type="submit">Iniciar Sesión</button>
      </form>
      <hr>
      <h2>Registrar Nuevo Usuario</h2>
      <form id="register-form">
        <input type="text" id="reg-username" placeholder="Nuevo Usuario"
required>
        <input type="password" id="reg-password" placeholder="Nueva
Contraseña" required>
        <button type="submit">Registrar</button>
      </form>
    </div>

    <div class="app-section hidden" id="app-section">
      <div class="section-container">
        <div class="form-section">
          <h3>Catálogo de Libros</h3>
          <button id="get-books">Obtener Todos los Libros</button>
          <div id="books-container"></div>
          <hr>

          <h3>Buscar por ISBN</h3>
```

```

        <form id="search-isbn-form" class="search-form">
            <input type="text" id="isbn-search" placeholder="ISBN del
libro" required>
            <button type="submit">Buscar por ISBN</button>
        </form>

        <h3>Buscar por autor</h3>
        <form id="search-author-form" class="search-form">
            <input type="text" id="author-search" placeholder="Nombre del
autor" required>
            <button type="submit">Buscar por Autor</button>
        </form>

        <h3>Buscar por formato</h3>
        <form id="search-format-form" class="search-form">
            <input type="text" id="format-search" placeholder="Formato
(e.g., Paperback)" required>
            <button type="submit">Buscar por Formato</button>
        </form>
    </div>

    <div class="form-section">
        <h3>Insertar libro</h3>
        <form id="insert-form" class="crud-form">
            <input type="text" id="isbn-insert" placeholder="ISBN"
required>
            <input type="text" id="titulo-insert" placeholder="Título"
required>
            <input type="number" id="id_autor-insert" placeholder="ID
Autor" required>
            <input type="number" id="id_categoria-insert" placeholder="ID
Categoría" required>
            <input type="number" id="id_editorial-insert" placeholder="ID
Editorial" required>
            <input type="number" id="anio_publicacion-insert"
placeholder="Año Publicación" required>
            <input type="number" id="price-insert" placeholder="Precio"
step="0.01" required>
            <input type="number" id="stock-insert" placeholder="Stock"
required>
            <input type="text" id="formato-insert" placeholder="Formato"
required>
            <button type="submit">Insertar Libro</button>
        </form>

        <h3>Actualizar libro</h3>
        <form id="update-form" class="crud-form">
            <input type="text" id="isbn-update" placeholder="ISBN a
actualizar" required>
            <input type="text" id="titulo-update" placeholder="Nuevo
Título">

```



```

        <input type="number" id="id_autor-update" placeholder="Nuevo
ID Autor">
        <input type="number" id="id_categoria-update"
placeholder="Nuevo ID Categoría">
        <input type="number" id="id_editorial-update"
placeholder="Nuevo ID Editorial">
        <input type="number" id="anio_publicacion-update"
placeholder="Nuevo Año Publicación">
        <input type="number" id="price-update" placeholder="Nuevo
Precio" step="0.01">
        <input type="number" id="stock-update" placeholder="Nuevo
Stock">
        <input type="text" id="formato-update" placeholder="Nuevo
Formato">
        <button type="submit">Actualizar Libro</button>
    </form>

    <h3>Eliminar libro</h3>
    <form id="delete-form" class="crud-form">
        <input type="text" id="isbn-delete" placeholder="ISBN a
eliminar" required>
        <button type="submit">Eliminar Libro</button>
    </form>
</div>
</div>
</div>

<div class="log-section">
    <h2>Logs del Cliente</h2>
    <pre id="client-logs"></pre>
</div>
</div>

<script src="script.js"></script>
</body>
</html>

```

Código **style.css**:

```

CSS
body {
    font-family: 'Arial', sans-serif;
    background-color: #f0f2f5;
    color: #333;
    display: flex;
    justify-content: center;
    align-items: flex-start;
    min-height: 100vh;
    margin: 0;
    padding: 20px;
}

```

```

}

.container {
  background: #fff;
  padding: 40px;
  border-radius: 12px;
  box-shadow: 0 6px 15px rgba(0, 0, 0, 0.1);
  width: 100%;
  max-width: 1200px;
  box-sizing: border-box;
}

h1, h2, h3 {
  color: #0056b3;
  text-align: center;
  margin-bottom: 20px;
}

h1 {
  font-size: 2.5em;
  border-bottom: 2px solid #0056b3;
  padding-bottom: 10px;
  margin-bottom: 30px;
}

h2 {
  font-size: 1.8em;
}

h3 {
  font-size: 1.4em;
  margin-top: 30px;
  margin-bottom: 15px;
}

.auth-section, .app-section {
  margin-bottom: 30px;
  padding: 20px;
  border: 1px solid #ddd;
  border-radius: 8px;
}

.hidden {
  display: none;
}

/* Flexbox para alinear las secciones */
.section-container {
  display: flex;
  gap: 30px; /* Espacio entre las columnas */
  flex-wrap: wrap; /* Permite que las columnas se envuelvan en pantallas pequeñas */
}

```

```

.form-section {
  flex: 1; /* Hace que cada sección ocupe el mismo espacio */
  min-width: 300px;
  padding: 20px;
  border: 1px solid #eee;
  border-radius: 8px;
  background-color: #f9f9f9;
}

.search-form, .crud-form {
  display: flex;
  flex-direction: column;
  gap: 10px;
  margin-bottom: 20px;
}

input[type="text"],
input[type="password"],
input[type="number"] {
  width: 100%;
  padding: 12px;
  border-radius: 6px;
  border: 1px solid #ccc;
  box-sizing: border-box;
  font-size: 1em;
}

button {
  background-color: #007bff;
  color: white;
  font-weight: bold;
  padding: 12px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background-color 0.3s ease;
  font-size: 1.1em;
}

button:hover {
  background-color: #0056b3;
}

hr {
  border: 0;
  height: 1px;
  background-image: linear-gradient(to right, rgba(0, 0, 0, 0), rgba(0, 0, 0, 0.25),
  rgba(0, 0, 0, 0));
  margin: 40px 0;
}

```

```

#books-container {
  margin-top: 25px;
  padding: 20px;
  border: 1px solid #eee;
  border-radius: 8px;
  background-color: #fafafa;
}

.book-item {
  background-color: #e9ecef;
  padding: 15px;
  border-radius: 6px;
  margin-bottom: 10px;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.05);
}

#client-logs {
  background-color: #2c2c2c;
  color: #00ff00;
  padding: 15px;
  border-radius: 8px;
  white-space: pre-wrap;
  font-family: 'Courier New', Courier, monospace;
  max-height: 250px;
  overflow-y: scroll;
  line-height: 1.4;
}

```

Código **script.js**:

```

JavaScript
const API_URL = 'http://35.222.91.194:5000';

const loginForm = document.getElementById('login-form');
const registerForm = document.getElementById('register-form');
const appSection = document.getElementById('app-section');
const getBooksBtn = document.getElementById('get-books');
const booksContainer = document.getElementById('books-container');
const clientLogs = document.getElementById('client-logs');

// Nuevos selectores para las búsquedas
const searchIsbnForm = document.getElementById('search-isbn-form');
const searchFormatForm = document.getElementById('search-format-form');
const searchAuthorForm = document.getElementById('search-author-form');

// Selectores para operaciones CRUD
const insertForm = document.getElementById('insert-form');
const updateForm = document.getElementById('update-form');
const deleteForm = document.getElementById('delete-form');

```

```

// Función para agregar logs al área de logs del cliente
const log = (message, type = 'info') => {
  const timestamp = new Date().toLocaleTimeString();
  clientLogs.innerHTML += `[${timestamp}] [${type.toUpperCase()}] ${message}\n`;
  clientLogs.scrollTop = clientLogs.scrollHeight;
};

// --- Flujo de Autenticación y Token Handling ---

// 1. Manejar el inicio de sesión
loginForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  const username = document.getElementById('username').value;
  const password = document.getElementById('password').value;

  log('Intentando iniciar sesión...', 'info');
  try {
    const response = await fetch(`${API_URL}/login`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ username, password }),
    });

    const data = await response.json();

    if (response.ok) {
      log('Inicio de sesión exitoso!', 'success');
      log('Tokens recibidos. Guardando en localStorage.', 'info');
      // Guardar tokens en el almacenamiento local del navegador
      localStorage.setItem('access_token', data.access_token);
      localStorage.setItem('refresh_token', data.refresh_token);

      // Ocultar la sección de login y mostrar la de la app
      document.querySelector('.auth-section').classList.add('hidden');
      appSection.classList.remove('hidden');
    } else {
      log(`Error de autenticación: ${data.msg}`, 'error');
    }
  } catch (error) {
    log(`Error de red al intentar iniciar sesión: ${error.message}`, 'error');
  }
});

// 2. Manejar el registro
registerForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  const username = document.getElementById('reg-username').value;
  const password = document.getElementById('reg-password').value;

```

```

log('Intentando registrar nuevo usuario...', 'info');
try {
  const response = await fetch(`${API_URL}/register`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ username, password }),
  });

  const data = await response.json();

  if (response.ok) {
    log('¡Registro exitoso!', 'success');
    log('Ahora puedes iniciar sesión con tu nuevo usuario.', 'info');
  } else {
    log(`Error de registro: ${data.msg}`, 'error');
  }
} catch (error) {
  log(`Error de red al intentar registrar: ${error.message}`, 'error');
}
});

// 3. Manejar la solicitud de libros con el token
getBooksBtn.addEventListener('click', async () => {
  log('Intentando obtener la lista de libros...', 'info');

  const accessToken = localStorage.getItem('access_token');
  if (!accessToken) {
    log('Error: No se encontró el token de acceso. Por favor, inicie sesión.', 'error');
    return;
  }

  try {
    const response = await fetch(`${API_URL}/api/books`, {
      method: 'GET',
      headers: {
        'Authorization': `Bearer ${accessToken}`,
      },
    });

    if (response.ok) {
      log('¡Solicitud exitosa!', 'success');
      // Parsear el XML
      const xmlText = await response.text();
      const parser = new DOMParser();
      const xmlDoc = parser.parseFromString(xmlText, "text/xml");

      // Mostrar los libros
      displayBooks(xmlDoc);
    } else if (response.status === 401) {

```

```

        log('Token expirado o inválido. Intentando refrescar el token...',
'warning');
        await refreshToken();
        // Reintentar la solicitud después de refrescar
        await getBooksBtn.click();
    } else {
        const errorText = await response.text();
        log(`Error al obtener libros: ${response.status} - ${errorText}`,
'error');
    }
} catch (error) {
    log(`Error de red al obtener libros: ${error.message}`, 'error');
}
});

// 4. Manejar el refresco del token
const refreshToken = async () => {
    const refreshToken = localStorage.getItem('refresh_token');
    if (!refreshToken) {
        log('Error: No se encontró el token de refresco. Por favor, vuelva a iniciar
sesión.', 'error');
        // Redirigir al usuario al login
        document.querySelector('.auth-section').classList.remove('hidden');
        appSection.classList.add('hidden');
        return;
    }

    try {
        const response = await fetch(`${API_URL}/refresh`, {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${refreshToken}`,
            },
        });

        const data = await response.json();

        if (response.ok) {
            log('Token de acceso refrescado con éxito.', 'success');
            localStorage.setItem('access_token', data.access_token);
            log('Nuevo token de acceso guardado.', 'info');
        } else {
            log(`Error al refrescar el token: ${data.msg}. Sesión expirada.`,
'error');
            // Redirigir al usuario al login
            document.querySelector('.auth-section').classList.remove('hidden');
            appSection.classList.add('hidden');
        }
    } catch (error) {
        log(`Error de red al intentar refrescar el token: ${error.message}`, 'error');
    }
};

```

```
// --- Funciones para manejar la UI ---

function displayBooks(xmlDoc) {
  booksContainer.innerHTML = ''; // Limpiar el contenedor
  const books = xmlDoc.getElementsByTagName('book');
  if (books.length === 0) {
    booksContainer.innerHTML = '<p>No se encontraron libros.</p>';
    return;
  }

  for (const book of books) {
    const isbn = book.getAttribute('isbn');
    const title = book.getElementsByTagName('title')[0]?.textContent;
    const author = book.getElementsByTagName('author')[0]?.textContent;
    const year = book.getElementsByTagName('year')[0]?.textContent;
    const genre = book.getElementsByTagName('genre')[0]?.textContent;
    const price = book.getElementsByTagName('price')[0]?.textContent;
    const stock = book.getElementsByTagName('stock')[0]?.textContent;
    const format = book.getElementsByTagName('format')[0]?.textContent;

    const bookDiv = document.createElement('div');
    bookDiv.classList.add('book-item');
    bookDiv.innerHTML = `
      <strong>ISBN:</strong> ${isbn || 'Desconocido'}<br>
      <strong>Título:</strong> ${title || 'Desconocido'}<br>
      <strong>Autor:</strong> ${author || 'Desconocido'}<br>
      <strong>Año:</strong> ${year || 'Desconocido'}<br>
      <strong>Género:</strong> ${genre || 'Desconocido'}<br>
      <strong>Precio:</strong> ${price || 'Desconocido'}<br>
      <strong>Stock:</strong> ${stock || 'Desconocido'}<br>
      <strong>Formato:</strong> ${format || 'Desconocido'}
    `;
    booksContainer.appendChild(bookDiv);
  }
}

// --- Manejar el insert ---
insertForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  const accessToken = localStorage.getItem('access_token');
  if (!accessToken) {
    log('Error: No se encontró el token de acceso. Inicie sesión.', 'error');
    return;
  }

  const bookData = {
    isbn: document.getElementById('isbn-insert').value,
    titulo: document.getElementById('titulo-insert').value,
    id_autor: parseInt(document.getElementById('id_autor-insert').value),
    id_categoria: parseInt(document.getElementById('id_categoria-insert').value),
    id_editorial: parseInt(document.getElementById('id_editorial-insert').value),
  };

```



```

        anio_publicacion:
    parseInt(document.getElementById('anio_publicacion-insert').value),
    price: parseFloat(document.getElementById('price-insert').value),
    stock: parseInt(document.getElementById('stock-insert').value),
    formato: document.getElementById('formato-insert').value,
  });

  log('Intentando insertar un nuevo libro...', 'info');
  try {
    const response = await fetch(`${API_URL}/api/books/book/insert`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${accessToken}`,
      },
      body: JSON.stringify(bookData),
    });

    const text = await response.text();
    if (response.ok) {
      log('Libro insertado con éxito!', 'success');
    } else {
      log(`Error al insertar libro: ${response.status} - ${text}`, 'error');
    }
  } catch (error) {
    log(`Error de red al insertar libro: ${error.message}`, 'error');
  }
});

// --- Manejar el update ---
updateForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  const accessToken = localStorage.getItem('access_token');
  if (!accessToken) {
    log('Error: No se encontró el token de acceso. Inicie sesión.', 'error');
    return;
  }

  const bookData = {
    isbn: document.getElementById('isbn-update').value,
    titulo: document.getElementById('titulo-update').value,
    id_autor: document.getElementById('id_autor-update').value,
    id_categoria: document.getElementById('id_categoria-update').value,
    id_editorial: document.getElementById('id_editorial-update').value,
    anio_publicacion: document.getElementById('anio_publicacion-update').value,
    price: document.getElementById('price-update').value,
    stock: document.getElementById('stock-update').value,
    formato: document.getElementById('formato-update').value,
  };

  // Filtrar los campos vacíos para no enviar datos nulos

```

```

    Object.keys(bookData).forEach(key => bookData[key] === '' && delete
bookData[key]);

    log('Intentando actualizar un libro...', 'info');
    try {
        const response = await fetch(`${API_URL}/api/books/book/update`, {
            method: 'PUT',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${accessToken}`,
            },
            body: JSON.stringify(bookData),
        });

        const text = await response.text();
        if (response.ok) {
            log('Libro actualizado con éxito!', 'success');
        } else {
            log(`Error al actualizar libro: ${response.status} - ${text}`, 'error');
        }
    } catch (error) {
        log(`Error de red al actualizar libro: ${error.message}`, 'error');
    }
});

// --- Manejar el delete ---
deleteForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    const accessToken = localStorage.getItem('access_token');
    if (!accessToken) {
        log('Error: No se encontró el token de acceso. Inicie sesión.', 'error');
        return;
    }

    const isbn = document.getElementById('isbn-delete').value;
    log('Intentando eliminar un libro...', 'info');
    try {
        const response = await fetch(`${API_URL}/api/books/book/delete?isbn=${isbn}`,
{
            method: 'DELETE',
            headers: {
                'Authorization': `Bearer ${accessToken}`,
            },
        });

        const text = await response.text();
        if (response.ok) {
            log('Libro eliminado con éxito!', 'success');
        } else {
            log(`Error al eliminar libro: ${response.status} - ${text}`, 'error');
        }
    } catch (error) {

```

```

        log(`Error de red al eliminar libro: ${error.message}`, 'error');
    }
});

// --- Manejar las búsquedas (GET) ---

// Buscar por ISBN
searchIsbnForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    const accessToken = localStorage.getItem('access_token');
    const isbn = document.getElementById('isbn-search').value;
    if (!accessToken) { log(`Error: No se encontró el token de acceso.`, 'error');
return; }

    log(`Buscando libro con ISBN: ${isbn}...`, 'info');
    try {
        const response = await fetch(`${API_URL}/api/books/${isbn}`, {
            method: 'GET',
            headers: { 'Authorization': `Bearer ${accessToken}` },
        });
        const xmlText = await response.text();
        if (response.ok) {
            const parser = new DOMParser();
            const xmlDoc = parser.parseFromString(xmlText, "text/xml");
            displayBooks(xmlDoc);
            log(`Búsqueda por ISBN exitosa.`, 'success');
        } else {
            log(`Error en la búsqueda por ISBN: ${response.status} - ${xmlText}`,
'error');
        }
    } catch (error) { log(`Error de red en la búsqueda por ISBN: ${error.message}`,
'error'); }
});

// Buscar por formato
searchFormatForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    const accessToken = localStorage.getItem('access_token');
    const format = document.getElementById('format-search').value;
    if (!accessToken) { log(`Error: No se encontró el token de acceso.`, 'error');
return; }

    log(`Buscando libros con formato: ${format}...`, 'info');
    try {
        const response = await fetch(`${API_URL}/api/books/formats/${format}`, {
            method: 'GET',
            headers: { 'Authorization': `Bearer ${accessToken}` },
        });
        const xmlText = await response.text();
        if (response.ok) {
            const parser = new DOMParser();
            const xmlDoc = parser.parseFromString(xmlText, "text/xml");

```

```

        displayBooks(xmlDoc);
        log(`Búsqueda por formato exitosa.`, 'success');
    } else {
        log(`Error en la búsqueda por formato: ${response.status} - ${xmlText}`,
        'error');
    }
    } catch (error) { log(`Error de red en la búsqueda por formato: ${error.message}`,
    'error'); }
    });

// Buscar por autor
searchAuthorForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    const accessToken = localStorage.getItem('access_token');
    const author = document.getElementById('author-search').value;
    if (!accessToken) { log(`Error: No se encontró el token de acceso.`, 'error');
    return; }

    log(`Buscando libros por autor: ${author}...`, 'info');
    try {
        const response = await fetch(`${API_URL}/api/books/author/${author}`, {
            method: 'GET',
            headers: { 'Authorization': `Bearer ${accessToken}` },
        });
        const xmlText = await response.text();
        if (response.ok) {
            const parser = new DOMParser();
            const xmlDoc = parser.parseFromString(xmlText, "text/xml");
            displayBooks(xmlDoc);
            log(`Búsqueda por autor exitosa.`, 'success');
        } else {
            log(`Error en la búsqueda por autor: ${response.status} - ${xmlText}`,
            'error');
        }
    } catch (error) { log(`Error de red en la búsqueda por autor: ${error.message}`,
    'error'); }
    });

```

Estos tres códigos se copiaron y pegaron en sus respectivos archivos, con el mismo nombre, en el nuevo directorio *WebApplication* el cual está separado y aislado del directorio donde se encuentran los microservicios.

Luego se tuvo que crear otra regla de firewall para que el navegador pudiera mostrar la página web sin que saliera error de conexión o acceso, dicha regla es la siguiente:

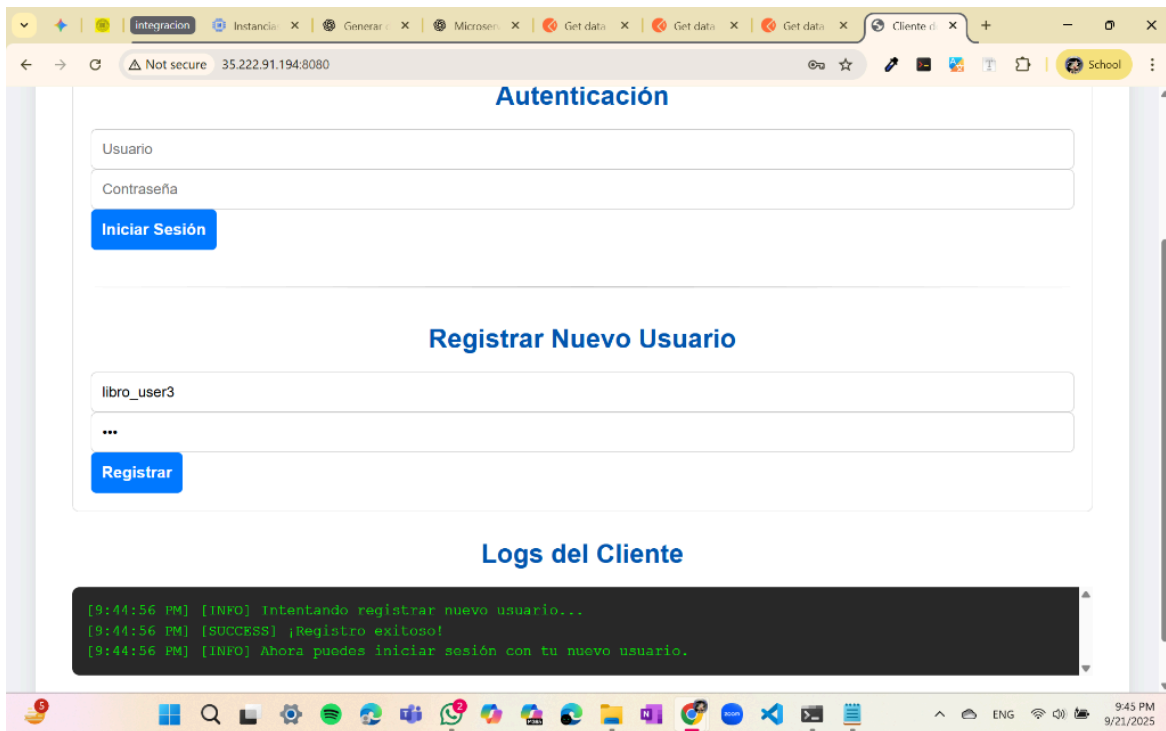
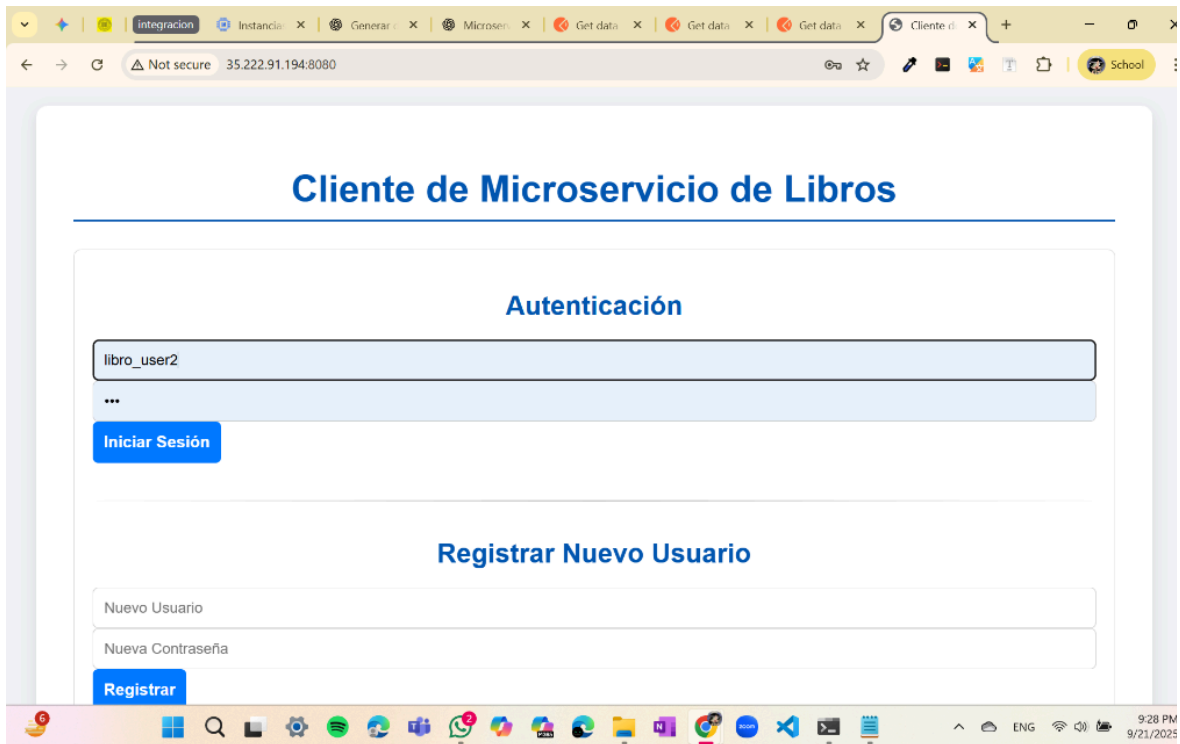
```
C:\Users\mccco>gcloud compute firewall-rules create allow-http-web-app --network=default --direction=INGRESS --priority=1000 --action=ALLOW --rules=tcp:8080 --source-ranges=0.0.0.0/0
Creating firewall...Created [https://www.googleapis.com/compute/v1/projects/integracion-468300/global/firewalls/allow-http-web-app].
Creating firewall...done.
NAME          NETWORK  DIRECTION  PRIORITY  ALLOW  DENY  DISABLED
allow-http-web-app  default  INGRESS    1000      tcp:8080      False
C:\Users\mccco>
```

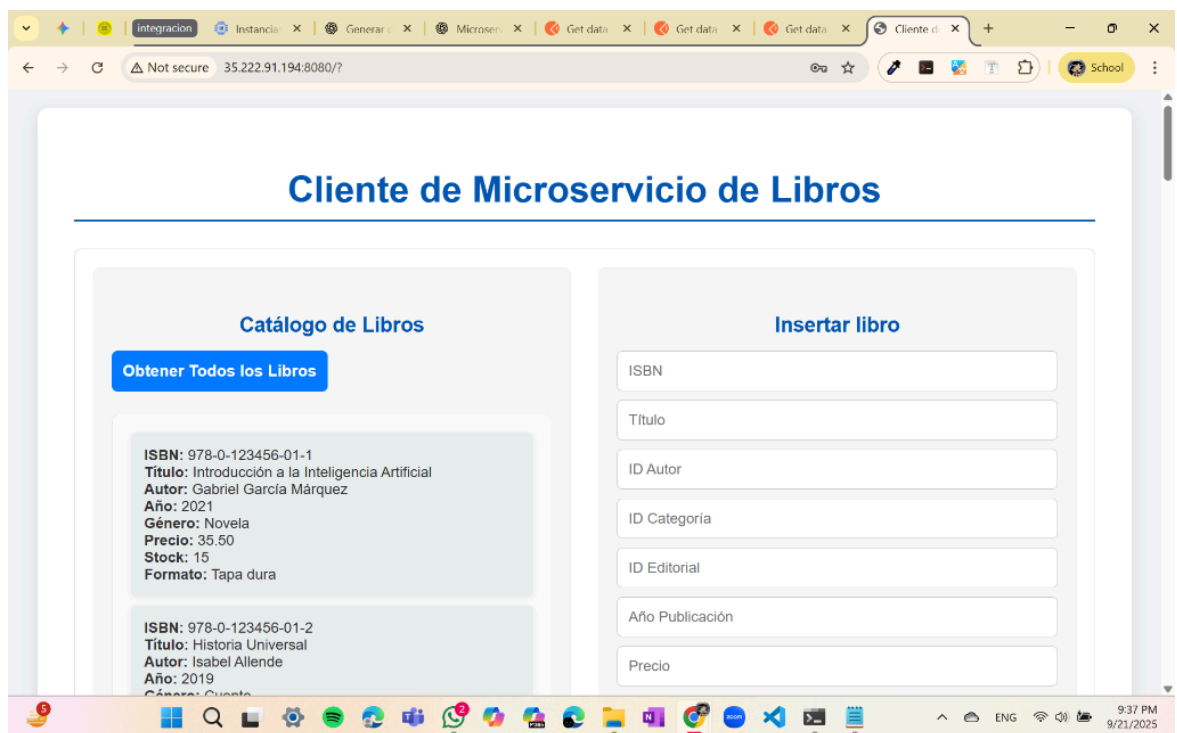
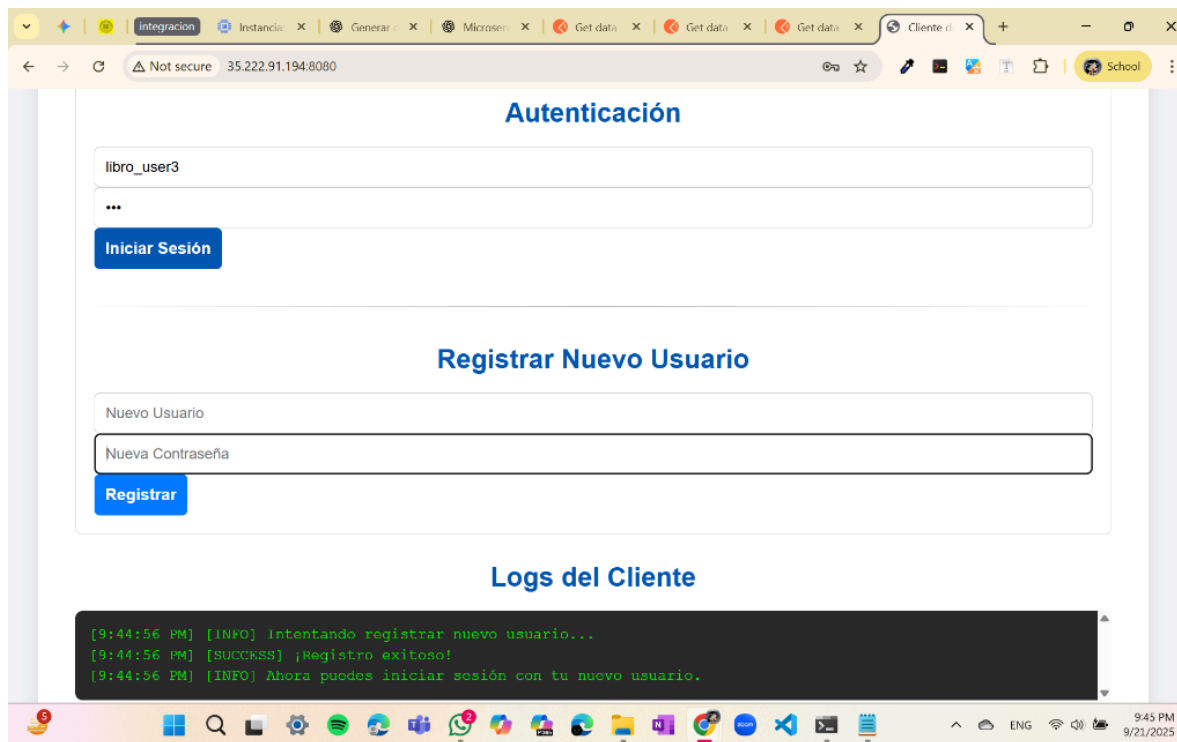
Para ejecutar la página web, se tiene que ejecutar el microservicio dentro de su directorio *Microservices/micro01* con los comandos **export FLASK_APP=main.py** y **flask run --host=0.0.0.0**. Dentro del directorio de WebApplication, se escribe el comando **sudo python3 -m http.server 8080**.

```
Installing collected packages: Flask-Cors
Successfully installed Flask-Cors-6.0.1
[m03681557@maquina01 micro01]$ export FLASK_APP=main.py
[m03681557@maquina01 micro01]$ flask run --host=0.0.0.0
 * Serving Flask app 'main.py'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.128.0.3:5000
Press CTRL+C to quit
```

```
[m03681557@maquina01 WebApplication]$ sudo python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
201.156.33.99 - - [22/Sep/2025 03:27:58] "GET / HTTP/1.1" 304 -
201.156.33.99 - - [22/Sep/2025 03:27:58] "GET /style.css HTTP/1.1" 304 -
201.156.33.99 - - [22/Sep/2025 03:27:58] "GET /script.js HTTP/1.1" 200 -
□
```

Por último, se muestran fotos del funcionamiento de la página web resultante:





Catálogo de Libros

Obtener Todos los Libros

ISBN: 978-0-123456-02-5
Título: Psicología Positiva
Autor: Federico García Lorca
Año: 2018
Género: Teatro
Precio: 25.60
Stock: 16
Formato: Tapa blanda

Buscar por ISBN

978-0-123456-02-5

Buscar por ISBN

Insertar libro

ISBN

Título

ID Autor

ID Categoría

ID Editorial

Año Publicación

Precio

Stock

Formato

Insertar Libro

Logs del Cliente

```
[9:37:40 PM] [INFO] Intentando iniciar sesión...
[9:37:41 PM] [SUCCESS] ¡Inicio de sesión exitoso!
[9:37:41 PM] [INFO] Tokens recibidos. Guardando en localStorage.
[9:37:42 PM] [INFO] Intentando obtener la lista de libros...
[9:37:42 PM] [SUCCESS] ¡Solicitud exitosa!
[9:38:07 PM] [INFO] Buscando libro con ISBN: 978-0-123456-02-5...
[9:38:07 PM] [SUCCESS] Búsqueda por ISBN exitosa.
```


integración x Instancia x Generar x Microsen x Get data x Get data x Get data x Cliente d x +

← → ↻ ⚠ Not secure 35.222.91.194:8080/? ☆ 🔧 🖨 📄 📱 📧 School

ISBN: 978-0-123456-02-2
 Título: Física Moderna
 Autor: Laura Esquivel
 Año: 2021
 Género: Ensayo
 Precio: 45.20
 Stock: 11
 Formato: Digital

Título

ID Autor

ID Categoría

ID Editorial

Año Publicación

Precio

Stock

Formato

Insertar Libro

Actualizar libro

ISBN a actualizar

Nuevo Título

Buscar por ISBN

ISBN del libro

Buscar por ISBN

Buscar por autor

Laura Esquivel

Buscar por Autor

Logs del Cliente

```

[9:37:40 PM] [INFO] Intentando iniciar sesión...
[9:37:41 PM] [SUCCESS] ¡Inicio de sesión exitoso!
[9:37:41 PM] [INFO] Tokens recibidos. Guardando en localStorage.
[9:37:42 PM] [INFO] Intentando obtener la lista de libros...
[9:37:42 PM] [SUCCESS] ¡Solicitud exitosa!
[9:38:07 PM] [INFO] Buscando libro con ISBN: 978-0-123456-02-5...
[9:38:07 PM] [SUCCESS] Búsqueda por ISBN exitosa.
[9:38:25 PM] [INFO] Intentando obtener la lista de libros...
[9:38:26 PM] [SUCCESS] ¡Solicitud exitosa!
[9:38:38 PM] [INFO] Buscando libros por autor: Laura Esquivel...
[9:38:39 PM] [SUCCESS] Búsqueda por autor exitosa.
  
```

Buscar por formato

Digital

Buscar por Formato

Logs del Cliente

```

[9:37:42 PM] [INFO] Intentando obtener la lista de libros...
[9:37:42 PM] [SUCCESS] ¡Solicitud exitosa!
[9:38:07 PM] [INFO] Buscando libro con ISBN: 978-0-123456-02-5...
[9:38:07 PM] [SUCCESS] Búsqueda por ISBN exitosa.
[9:38:25 PM] [INFO] Intentando obtener la lista de libros...
[9:38:26 PM] [SUCCESS] ¡Solicitud exitosa!
[9:38:38 PM] [INFO] Buscando libros por autor: Laura Esquivel...
[9:38:39 PM] [SUCCESS] Búsqueda por autor exitosa.
[9:38:55 PM] [INFO] Intentando obtener la lista de libros...
[9:38:55 PM] [SUCCESS] ¡Solicitud exitosa!
[9:39:06 PM] [INFO] Buscando libros con formato: Digital...
[9:39:07 PM] [SUCCESS] Búsqueda por formato exitosa.
  
```

ISBN: 978-0-123456-01-3
Título: El Viaje del Programador
Autor: Mario Vargas Llosa
Año: 2020
Género: Novela
Precio: 42.00
Stock: 12
Formato: Digital

ISBN: 978-0-123456-01-7
Título: Diseño UX/UI
Autor: Jorge Luis Borges
Año: 2023
Género: Novela
Precio: 48.75
Stock: 18
Formato: Digital

ISBN: 978-0-123456-02-2
Título: Física Moderna
Autor: Laura Esquivel
Año: 2021
Género: Ensayo
Precio: 45.20
Stock: 11
Formato: Digital

ISBN: 978-0-123456-01-4
Título: Marketing Digital
Autor: Miguel de Cervantes
Año: 2022
Género: Biografía

ID Autor

ID Categoría

ID Editorial

Año Publicación

Precio

Stock

Formato

Insertar Libro

Actualizar libro

ISBN a actualizar

Nuevo Título

Nuevo ID Autor

Catálogo de Libros

Obtener Todos los Libros

ISBN: 978-0-123456-01-1
Título: Introducción a la Inteligencia Artificial
Autor: Gabriel García Márquez
Año: 2021
Género: Novela
Precio: 35.50
Stock: 15
Formato: Tapa dura

ISBN: 978-0-123456-01-2
Título: Historia Universal
Autor: Isabel Allende
Año: 2019
Género: Cuento
Precio: 28.99
Stock: 8
Formato: Tapa blanda

ISBN: 978-0-123456-01-3
Título: El Viaje del Programador
Autor: Mario Vargas Llosa
Año: 2020
Género: Novela

Insertar libro

978-0321765723

The Lord of the Rings: The Fellowship of the Rin

1

1

1

1954

12.99

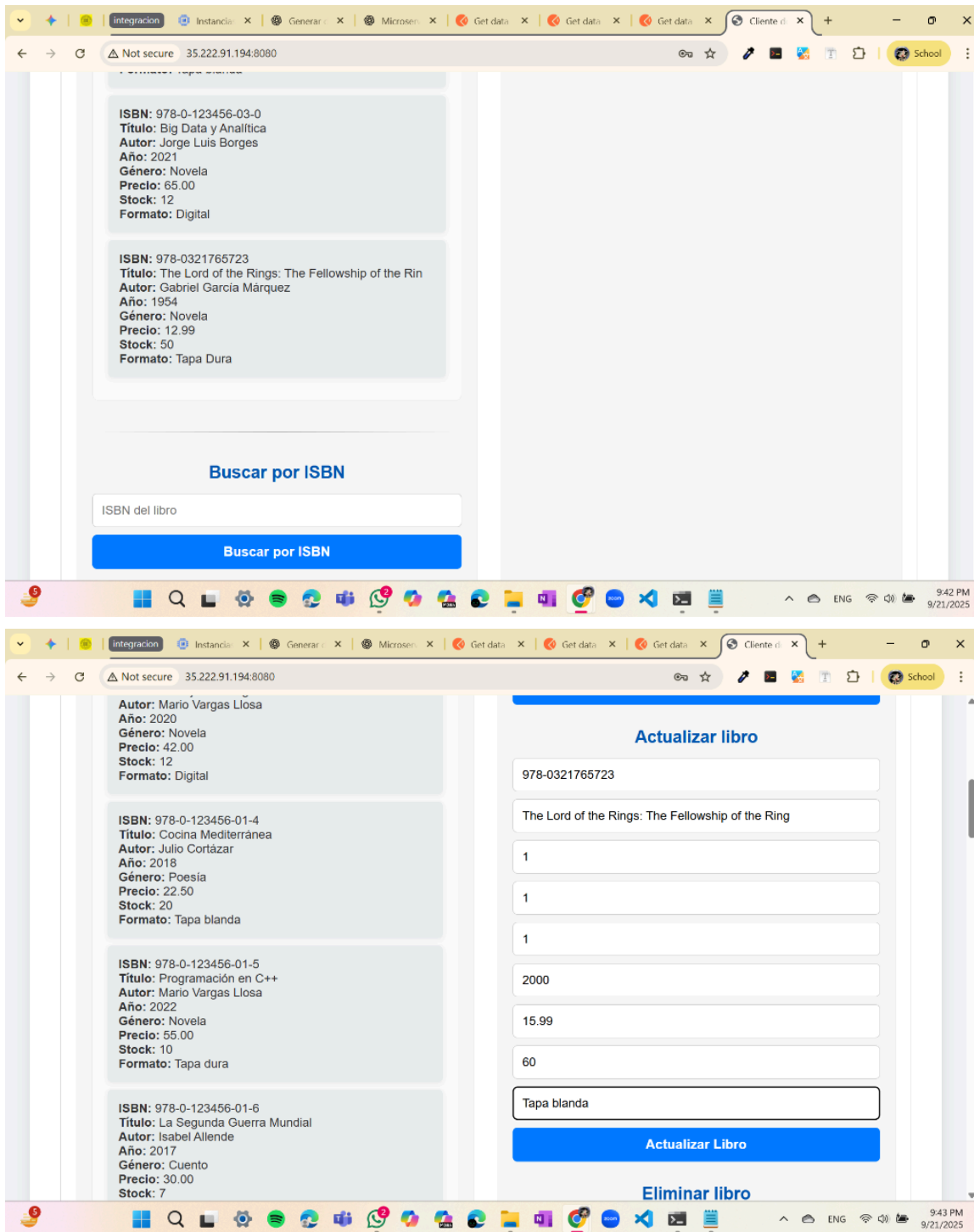
50

Tapa Dura

Insertar Libro

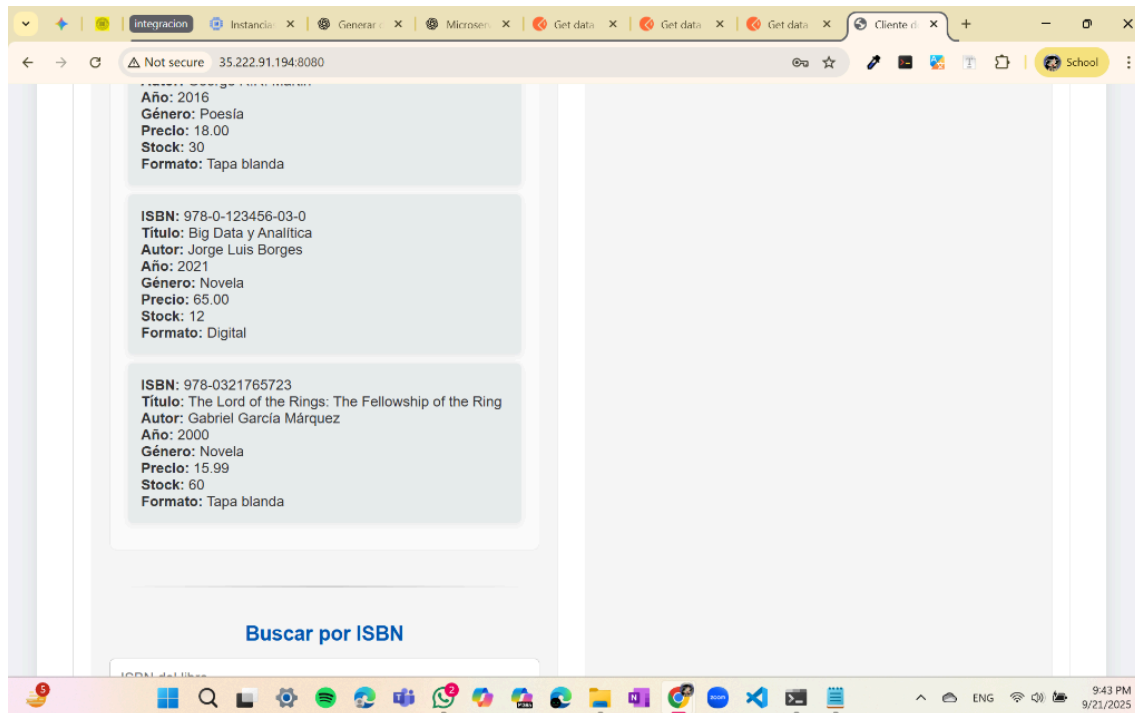
Logs del Cliente

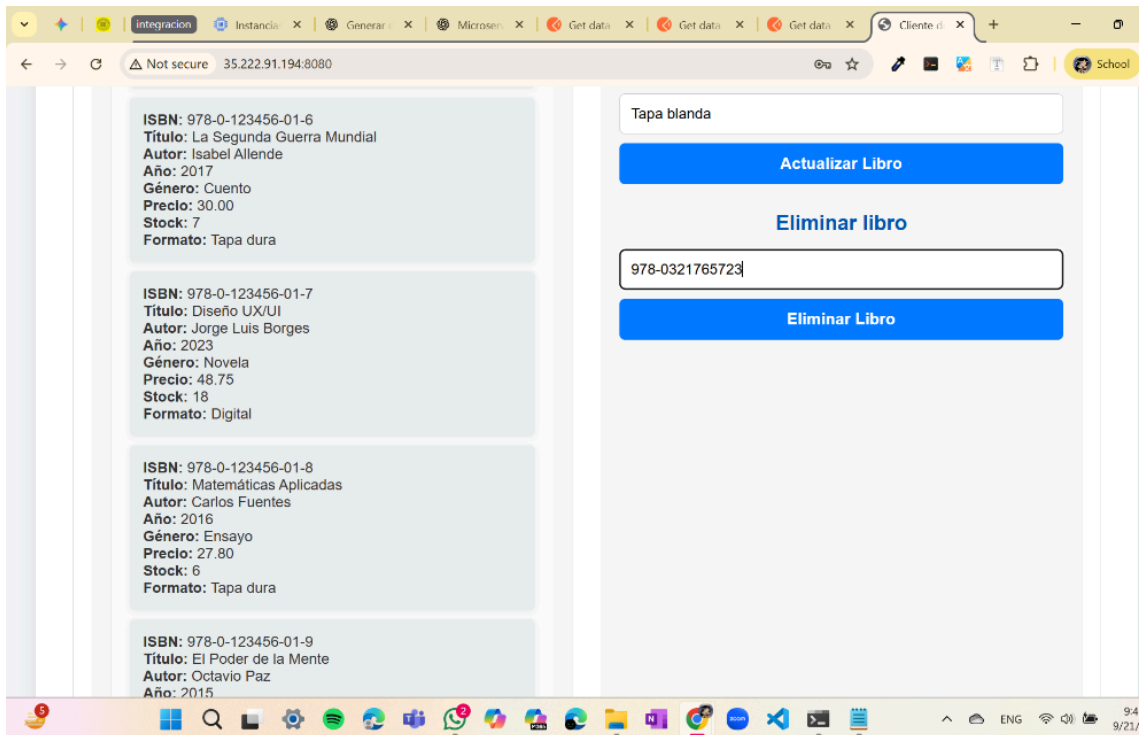
```
[9:39:50 PM] [INFO] Intentando iniciar sesión...
[9:39:51 PM] [SUCCESS] ¡Inicio de sesión exitoso!
[9:39:51 PM] [INFO] Tokens recibidos. Guardando en localStorage.
[9:39:52 PM] [INFO] Intentando obtener la lista de libros...
[9:39:53 PM] [SUCCESS] ¡Solicitud exitosa!
[9:41:32 PM] [INFO] Intentando insertar un nuevo libro...
[9:41:32 PM] [SUCCESS] Libro insertado con éxito!
```



Logs del Cliente

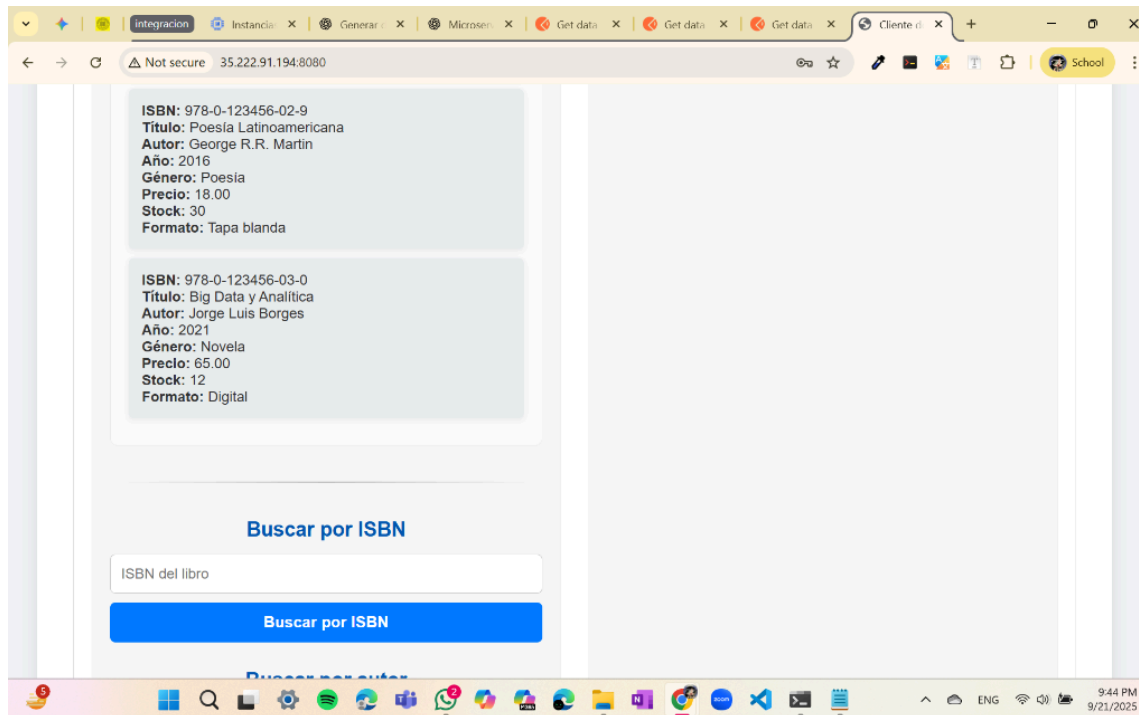
```
[9:39:52 PM] [INFO] Intentando obtener la lista de libros...
[9:39:53 PM] [SUCCESS] ¡Solicitud exitosa!
[9:41:32 PM] [INFO] Intentando insertar un nuevo libro...
[9:41:32 PM] [SUCCESS] Libro insertado con éxito!
[9:42:00 PM] [INFO] Intentando obtener la lista de libros...
[9:42:00 PM] [SUCCESS] ¡Solicitud exitosa!
[9:42:01 PM] [INFO] Intentando obtener la lista de libros...
[9:42:01 PM] [SUCCESS] ¡Solicitud exitosa!
[9:43:35 PM] [INFO] Intentando actualizar un libro...
[9:43:36 PM] [SUCCESS] Libro actualizado con éxito!
[9:43:41 PM] [INFO] Intentando obtener la lista de libros...
[9:43:41 PM] [SUCCESS] ¡Solicitud exitosa!
```





Logs del Cliente

```
[9:42:00 PM] [INFO] Intentando obtener la lista de libros...
[9:42:00 PM] [SUCCESS] ¡Solicitud exitosa!
[9:42:01 PM] [INFO] Intentando obtener la lista de libros...
[9:42:01 PM] [SUCCESS] ¡Solicitud exitosa!
[9:43:35 PM] [INFO] Intentando actualizar un libro...
[9:43:36 PM] [SUCCESS] Libro actualizado con éxito!
[9:43:41 PM] [INFO] Intentando obtener la lista de libros...
[9:43:41 PM] [SUCCESS] ¡Solicitud exitosa!
[9:44:16 PM] [INFO] Intentando eliminar un libro...
[9:44:17 PM] [SUCCESS] Libro eliminado con éxito!
[9:44:23 PM] [INFO] Intentando obtener la lista de libros...
[9:44:24 PM] [SUCCESS] ¡Solicitud exitosa!
```



Consideraciones importantes

1. La **estructura del directorio** del ejercicio es el siguiente:

/ (directorio raíz)

├── Microservices/

| ├── micro01/

| ├── main.py

└── WebApplication/

 ├── index.html

 ├── style.css

 └── script.js

2. Los **tokens** los **refresca en automático el microservicio cada 15 mins**. En caso de inactividad el usuario se sale del sitio web y solo se puede ingresar si el usuario ya está registrado e ingresa su nombre de usuario y contraseñas correctas. De lo contrario, no se puede acceder.
3. Se tuvo que **manipular el código del script.js** para que al mostrar los libros se vieran todos los atributos así como el **index.html** y **style.css** para que se mostraran de manera correcta y ordenada.

Conclusión

En resumen, esta actividad me permitió comprender de manera práctica cómo los microservicios, al operar de forma independiente, se integran para formar una aplicación completa. La implementación de un sistema de autenticación con JWT fue un aprendizaje clave, ya que me dio una visión integral de cómo se gestiona la seguridad en la comunicación entre un cliente y un servidor, asegurando que solo los usuarios autorizados puedan acceder a los recursos protegidos. Además, la experiencia de diseñar las operaciones CRUD (crear, leer, actualizar y eliminar) y manejar diferentes formatos de datos (XML para la consulta de libros y JSON para la autenticación y las operaciones) reforzó mi comprensión sobre la flexibilidad y eficiencia de las APIs.

Más allá de lo técnico, el ejercicio me ayudó a reforzar la importancia de la configuración del entorno de despliegue. Superar desafíos como la configuración de reglas de firewall en GCP y la solución de problemas de CORS (Cross-Origin Resource Sharing) me dio una visión más completa del ciclo de vida de una aplicación, desde el desarrollo hasta su puesta en marcha. Asimismo, trabajar con la IA como apoyo fue una herramienta invaluable para acelerar la productividad, aunque siempre requirió mi validación y adaptación para asegurar que el código cumpliera con los requisitos específicos del proyecto.

En conjunto, considero que esta práctica me dio un aprendizaje integral. La combinación de la creación del microservicio con Flask, la seguridad de JWT, la interacción con la base de datos MariaDB y la construcción del cliente web me proporcionó las habilidades esenciales para diseñar, implementar y gestionar arquitecturas de microservicios modernas, preparándome para desafíos más complejos en el futuro.